C Programming & Data Structures

PAGE NO.: 25
DATE: / /

16/20

C Programming	Data Structures
→ Variables	→ Stacks
→ Operators	→ Queues
→ Conditionals & loops	→ Linked lists
→ Functions	→ Trees
→ Recursion	→ Binary search trees
→ Pointers & arrays	→ Binary heaps
→ Structure & Union	→ Graphs

History
→ Machine language (in Binary 0,0110111)
→ Assembly (ADD A, B)
→ C

Why C?

Portability: Ability to run your code anywhere you want
Assembly: Not portable
C: Portable

Less lines of code

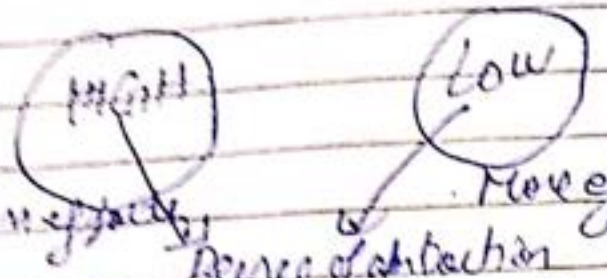
IN Assembly	IN C
Mov bl, 2	int result = 2+3;
Mov cl, 3	
Add bl, cl	
Mov result, bl	

Scanned with CamScanner

Features of C

1) Procedural language
2) High level language (machine high level low)

High level V/S Low level



COBOL, C, FORTRAN, C++, LISP, Pascal, Fortran

Assembly language

Why C middle level?

System Level Feature
a) Direct access to memory through pointer
b) Bit manipulation using bitwise operators
c) Writing assembly code within C code

3) Popular choice for system level apps
4) Wide variety of built in functions, standard libraries & header files.

stdio.h math.h string.h conio.h
stdlib.h time.h ctype.h.

scanf isdigit sqrt printf strcat

```
#include<stdio.h>
int main()
{
    printf("Nexo Academy");
}
```

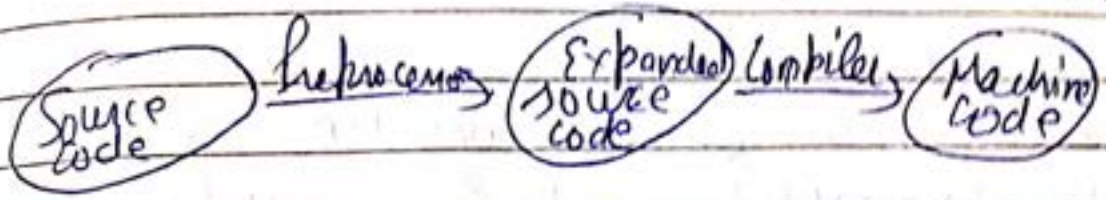
Scanned with CamScanner

Comment — //

PAGE NO: 257
DATE: / /

#include <stdio.h> → Preprocessor directive

Preprocessor: replaces text (starting with #)
with the actual content.
→ replaces before the compilation begins
→ output of preprocessing is expanded source code



#include<stdio.h> simply requests the preprocessor to include file stdio.h
→ header

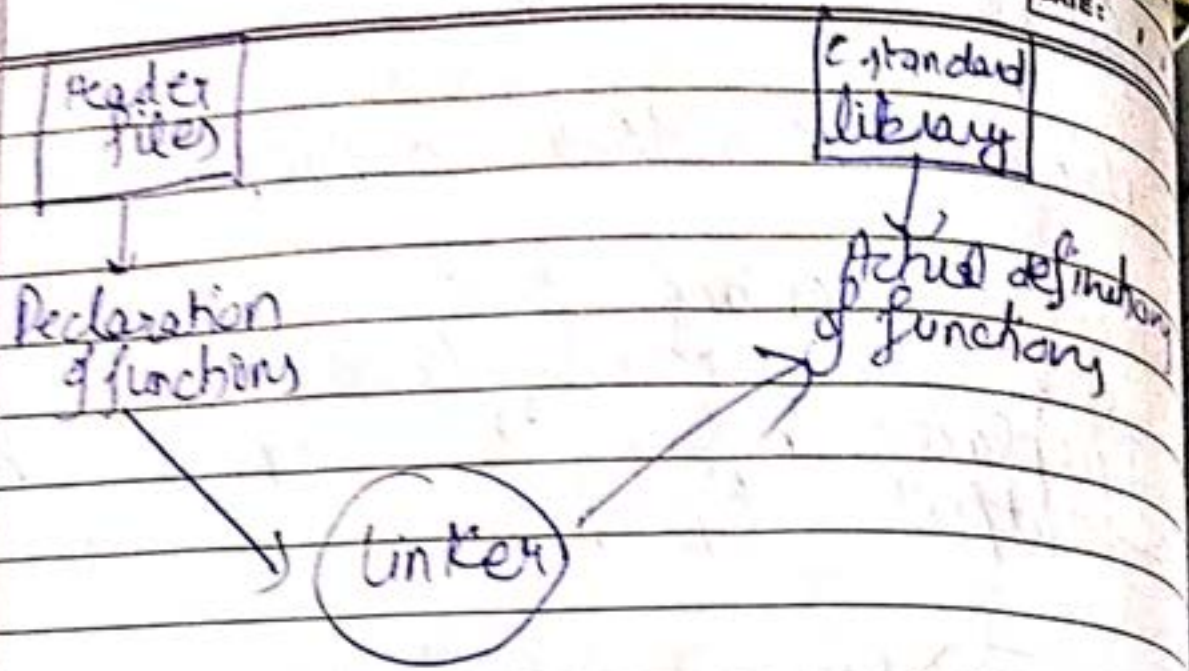
stdio.h : standard input output file
→ header file
→ contains declarations (prototypes) of functions like printf, scanf etc.

Syntax of a function

return_type name_of_function(parameter_type name_of_parameter, parameter_type ------)
{
set of statements;
}

Scanned with CamScanner

Why diff files?



∴ header files only contain declarations
→ Also we don't use preprocessor directive but linker because linker just links the code, but preprocessor will increase it to expanded size & C standard library has lot of code So compiler will take huge time to compile.

Hello word program!

```
#include <stdio.h>
int main()
{
printf("Hello world");
return 0;
}
```

Scanned with CamScanner

PAGE NO. 259
DATE :

Running hello world with following changes one by one

1) Comment #include<stdio.h>
Result → Error: printf was not declared in this scope

2) Remove semicolon after printf
Result → Error: Expected ';' before return

3) Remove int return type from main
Result → Hello World

4) return any integer of your choice
-ve integer : No Result but code executed
+ve " : Hello world

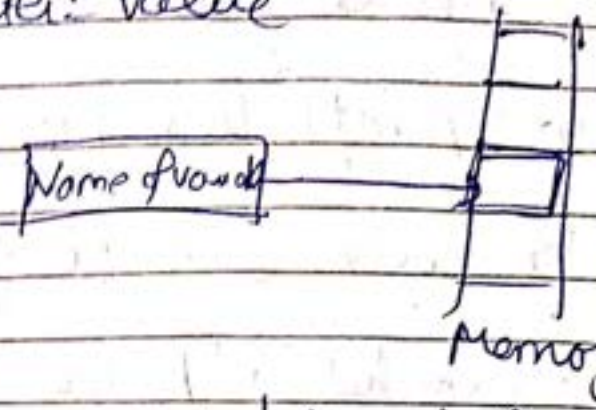
5) Replace 0 with Character, for eg('a')
No result but code executed

6) Remove semicolon after return
Result → Error: Expected ';' before '}' token

Scanned with CamScanner

Variables in C

Eg - 1 Gram = Variable
Water = Value



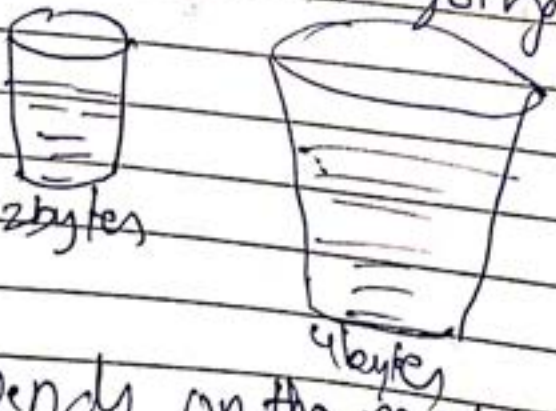
Note: Declare variables before use

Declaration: announcing the properties of variables to the compiler
properties
1. Size of the variable
2. Name of the "

Definition of a variable
Def: allocating memory to a variable

Most of the time declaration & definition will be done at the same time

Declaration
eg: int var;
↳ name of variable
↳ Datatype; how much space a variable is going to occupy in memory



Depends on the machine

Scanned with CamScanner

Note - 1) Code works without return 0 also
2) printf("%d\n", var1); //line change

PAGE NO.: 261
DATE: / /

Initialization

int var = 3;

Variable = vary - able

#include <stdio.h>
int main()
{
int var = 3; //all variables defined once (exception for same variable)
var = 4;
printf("%d", var);
return 0;
}

O/P → 4

We can also assign variable to variable

#include <stdio.h>
int main()
{ int var1 = 3;
int var2;
var2 = var1;
printf("%d", var2); O/P 4
return 0;
}

Defining variables in a single line

#include <stdio.h>
int main()
{
int var1, var2, var3;
var1 = var2 = var3 = 4; printf("%d", var1);
printf("%d", var2); printf("%d", var3); return 0; }

Scanned with CamScanner

Naming Conventions

PAGE NO.:
DATE:

Variable name: composed of letters or combination of letters (both uppercase & lowercase) & digits

Rule #1: don't start variable name with digit
global ✓ 2global ✗

Rule #2: beginning with underscore is valid but not recommended
_ (underscore) is treated as letter

Rule #3: C language is case sensitive. Uppercase letters are different from lowercase !!.
var, Var, VaR, vAR, vAr, VAR are diff.
Generally - lowercase (Variable name)
- Uppercase (Symbolic constant name)
↓
Use in Hashing

Rule #4: special character (@, #, $, ^, &, *, ---)
except underscore not allowed.
Sometimes $ can be accepted but not recommended

Scanned with CamScanner

Rule #5: Blank or white spaces are not allowed

int my variable; ✗ int my-variable; ✓

Rule # 6: Don't use keywords to name your variables

✗ - if, else, for, while, switch, int, float, long, double

✓ - If, Else, for, WHILE, Switch, INT, FloAt, LOngela

tip → Don't use looooonng ggg names for your variables

Basic O/P functions - printf

Ex: analyse printf ("Neso Academy");
printf ("%d", var);
printf ("%d %d", var1, var2);

conclusion

i) first argument always in double quotes

Ex: is not printed in but Neso academy is beca

%d is a placeholder for variable

d means decimal

2nd %d %d, var1, var2

1 space

printf("%d %d %d", val, val2);
error

#include <stdio.h>
int main() {
int two=2, three=3, six=6;
int result = (two + three) * six / three;
printf("I am now to programming but I know how to write authentic expressions like this: (%d + %d) * %d / %d", two, three, six, three);
printf("and the result of this expression is: %d", result);
}

Fundamental Data Types (integer)

Size of Integer - depends on MC (2, 4 bytes)

2 bytes = 16 bits
4 bytes = 32 bits
More the size more the capacity

Wanna know the size of Integer?

Use "sizeof" operator.

#include <stdio.h>
int main() {
printf("%d", sizeof(int));
return 0;
}

Note: (sizeof is a unary operator and not a function

Scanned with CamScanner

PAGE NO. 26
DATE: / /

sizeof(short int) = always 2
sizeof(long int) = ~~always~~ 4
sizeof(long long int) = 8
sizeof(long long int) = error
short short - error

Range: Upper & lower limit of some set of data
for eg: {0, 1, 2, 3, 4}
Range 0 to 4

Prerequisites

Decimal # system: Human understandable number system. Also called as base 10 number system
Range: 0 to 9

$10^2\ 10^1\ 10^0$
$568 = 8\times10^0 + 6\times10^1 + 5\times10^2$

Binary # system: Machine understandable number system. Also called as base 2 # system
Range: 0 to 1

$2^3\ 2^2\ 2^1\ 2^0$
1 1 0 0

$2^3\times1 + 2^2\times1 + 2^1\times0 + 2^0\times0 = 12$

Range of 4 bit data
0 0 0 0 Min value = 0
1 1 1 1 Max value = 15

Range: 0000 to 1111
0 to 15

Scanned with CamScanner

Formula to calculate max value = 2^n

Range of Integer (2 byte) [16 bits]
Unsigned range: 0 to 65535 (by applying: $2^n - 1$)
Signed range: -32768 to +32767
2's compliment range: $-(2^{n-1})$ to $+(2^{n-1} - 1)$

4 byte Unsigned range: 0 to 4294967295
(by applying: $2^n - 1$)
Signed range: -2147483648 to +2147483647

Modifiers (short, long, signed, unsigned)
used for integer

long & short - done

Note: By default int some_variable_name; is signed integer variable.

Unsigned int some_variable_name; allows only +ve values

Now let's do Range programatically

```
1) #include <stdio.h>          // For INT_MIN & INT_MAX
   #include <limits.h>
   int main()
   {
   int var1 = INT_MIN;          // These are symbolic constants
   int var2 = INT_MAX;
   printf("Range of signed integer is from: %d to %d", var1, var2);
   }
```

Scanned with CamScanner

PAGE NO.: 269
DATE: 7/9

```
#include<stdio.h>
#include<limits.h>
int main()
{
unsigned int var1 = 0;
unsigned int var2 = UINT_MAX;
printf("range of unsigned integer is from:
%ou to %ou", var1, var2);
}
```

No one is available in

placeholder Chars

```
#include<stdio.h>
#include<limits.h>
int main()
{
short int var1 = SHRT_MIN;
short int var2 = SHRT_MAX;
printf("range of short signed integer is
from: %od to %od", var1, var2);
}
```

```
#include<stdio.h>
,,,,<limits.h>
int main()
{
short unsigned int var1 = 0;
                  var2 = USHRT_MAX;
printf("range of short unsigned integer is
from: %ou to %ou", var1, var2);
}
```

short unsigned / unsigned short order does not matter

Note: if sizeof(long int) = 4 bytes
then sizeof(long long int) = 8 bytes
else
if sizeof(long int) = 8 bytes
then sizeof(long long int) = 8 bytes
: max limit also

Scanned with CamScanner

for long use LONG_MIN %ld
LONG_MAX %ld
unsigned 0 %lu
ULONG_MAX %lu

for long long signed LONG LONG_MIN %lld
LONG_LONG_MAX %lld
unsigned 0 %llu
ULONG_MAX %llu

Summary
1) sizeof(short) <= sizeof(int) <= sizeof(long)
2) %d → signed integer
3) %u → unsigned ''
4) %ld → long integer (signed)
5) %lu → unsigned long integer
6) %lld → long long integer
7) %llu → unsigned long integer

What if I exceed the range?
Let's do this
1. Exceeding the unsigned range
2. '' '' signed ''

1.
```
#include <stdio.h>
 '' , <limits.h>
int main() {
unsigned int var = UINT_MAX + 1;
printf("%d", var);
}
```

Scanned with CamScanner

Let's understand

3 bit unsigned range exceeding condition

PAGE NO.: 269
DATE:

2^2	2^1	2^0	
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

Let's exceed

8 = 1 0 0 0

So

8 = 0 0 0 0

Why 9 = 1 0 0 1 = 1

10 = 1 0 1 0 = 2

11 = 1 0 1 1 = 3

So it is MOD 8 or MOD 2^3 [illegible]

~~General MOD 2^n~~

for 32 bit unsigned data → Mod 2^{32}

for n " " → Mod 2^n

2.
```
#include<stdio.h>
#include<limits.h>
int main()
{ int var = INT_MAX
  var++;
  printf("%d", var) // -2147----
```

2nd data type - Char

ASCII values (requires 7 bits → till 127)

0 - 48

A - 65

a - 97

Scanned with CamScanner

Overview

char some_variable_name = 'N';
//only 1 variable at a time
∴ size = 1 byte

char some_variable_name = 65; //A

Inside printf, use
%c

```
#include <stdio.h>
int main()
{ char a = 'A';
  char a2 = 65;
printf("%c\n", a);
printf("%c\n", a2);
```

Size & range of character

Size = 1 byte = 8 bits

Range
Unsigned = 0 to 255
Signed = -128 to +127

Signed V/s Unsigned

signed

-128 = 1 0 0 0 0 0 0 0
-2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

+128 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0
1 0 0 0 0 0 0 0

Scanned with CamScanner

-2^7 6 5 4 3 2 1 2^0

PAGE NO.: 21
DATE:

127 = 1 0 0 0 0 0 0 1

2^7 2^0

129 1 0 0 0 0 0 0 1

0 → 127 Same value for signed & unsigned

Unsigned 128 = Signed -128

" 129 = " -127

,

,

,

255 = - - - -1

```
#include <stdio.h>
int main()
{
    char v1 = -128;
    char v2 = -128;
    printf("%c", v1);
    printf("%c", v2);
```

0 → 127 is known as traditional ASCII

128 → 255 is ... extended ASCII

(-128 → -1)

Scanned with CamScanner

→ Used to represent fractional or real numbers

Fundamental data types - float & double & long double

float → 4 bytes = 32 bits
Double → 8 bytes = 64 "
Long Double → 12 " = 96 "
Size of these data types totally depends from system to system

Representation
float → IEEE 754 Single precision floating point
Double → IEEE 754 Double " " "
Long " → Extended precision floating "

Overview on fixed & floating point

fixed point representation

eg
−9.99 → we cannot represent −7.9824 as it will be shown as −7.967 only
↓ ↓
sign Entire fraction

Min value = −9.99
Max " = +9.99

Floating point representation

eg
−0 + 99
↓ ↓ ↳ Mantissa
sign Exponent

Formula = (0.M) * Base^Expo
Min value = −(0.9) * 10^9
Max value = +(0.9 * 10^9)

Scanned with CamScanner

Why 3 diff data types?

PAGE NO.: 29
DATE: / /

```
#include <stdio.h>
int main()
{
float var1 = 3.14159265358979323;
double var2 = 3.14159265358979323;
long double var3 = 3.14159265358979323L;

printf("%d\n", sizeof(float));
printf("%d\n", sizeof(double));
printf("%d\n", sizeof(long double));
printf("%.16f\n", var1); //16 places after decimal
printf("%.16lf\n", var2);
printf("%.21Lf\n", var3);
}
```

```
4
8
12
```

: float precise upto 7 digits

Changed

3.1415927(41012573 2)

3.14159265358979(3) → precise upto 16 digits

3.14159265358979323(84626433832795) upto 19 digits

```
4
8
16
```

Same

Same

0-0000

%s - string

```
#include<stdio.h>
int main()
{ int var = 4/9;
printf("%d \n", var);

float var1 = 4/9;          // 0
printf("%.2f\n", var1);    // 0.00

float var2 = 4.0/9.0;      // 0.44
printf("%.2f \n", var2);
```

4.0/9.0 means double
4.0f/9.0f :, float

Questions Set-1

1) what is the o/p of following C program fragment:

```
#include<stdio.h>
int main()
{
printf("%d", printf("%s", "Hello World!"));
}
```

Ans. %s is used to print "string of characters" Inside printf not only prints the content on the screen. It also returns the # of characters that it successfully prints on screen

o/p Hello World! 12

Scanned with CamScanner

1. O/P of int main() {
printf("%10s", "Hello");
return 0; }

H e l l o
1 2 3 4 5 6 7 8 9 10
space

2. What is the O/P of following C program
int main() {
char c = 255;
c += 10;
printf("%d", c);
return 0;
}
a) 265
b) Something in ASCII table
c) 7
d) 9

d) 9 : %d if %c then b)

3. Which is Correct
signed int i;
signed i;
unsigned i;
long i;
long int i;
big long i;
a) I, V
b) I
c) All
d) IV, V, VI

c) All

In all cases where int not mentioned compiler implicitly assumes int

Scanned with CamScanner

Q5: What does following program print?

```
int main() {
    unsigned i = 1;
    int j = -4;
    printf("%ou", i+j);
    return 0;
}
```

a) garbage
b) -3
c) Integer value dependent from m/c
d) None

Ans- Before answering, let's see how -3 stored in 2's complement

-3 in in 2's Complement

Step 1 → Take 1's complement of 3

3 = 00000000 00000000 00000000 00000011

1's complement of 3 = 11111111 11111111 11111111 11111100

Step 2: Add 1 to result. It will give

11111111 11111111 11111111 11111101

= 4294967293

(on my computer = size 4 bytes)

Answer = -3 if %d instead of %ou

but Now %ou, and max value of unsigned = 4294967295

Our value = 4294967293

So ans = -2

Ans (C)

Scanned with CamScanner

Scope of variables

PAGE NO.: 27
DATE: / /

Scope - lifetime
The area under which a variable is applicable or alive

Strict Definition: a block or a region where a Variable is declared, defined and used, and when a block or a region ends, Variable is automatically destroyed.

Basic principle of Scoping

```
{
    ——> contents of outer block upto this point are visible to internal block
    {
    }  ——> contents of internal block are not visible to outer block
}

{
}  ——> Independent of each other
{
}
```

```
#include <stdio.h>
int main(){
  int var=3;
{ int var=4;
  printf("%d\n", var);
}
printf("%d", var);
return 0;
}
```

Scanned with CamScanner

```
#include <stdio.h>
int fun();
int var=10;  // global variable
int main() {
    int var=3;
    printf("%d \n", var); // 3
    fun();
    return 0;
}
int fun()
{
    printf("%d", var); // 10
}
```

Modifiers: auto & extern

auto means automatic
Variables declared inside a scope by default are automatic variables

Syntax: auto int some_variable_name;

```
#include<stdio.h>
int main(){
    int var;
    return 0;
}
```

=

```
#include<stdio.h>
int main(){
    auto int var;
    return 0;
}
```

auto are variables which destroys after scope ends. If you don't write auto, it itself assigns auto to it, hence above 2 are same

Scanned with CamScanner

Take aways

PAGE NO.: 279
DATE:

If you won't initialize auto variable, by default it will be initialized with some garbage (random) value

```
#include <stdio.h>
int main(){
  auto int var;
  printf("%d.d.", var);   //174
  return 0;
}
```

2) Other hand, global variable by default initialized to 0.

```
#include <stdio.h>
int var;
int main(){
  printf("%d", var);   //0
  return 0;
}
```

Extern modifier

int var; extern int var;

Declaration + Definition (Memory allotted) Declaration (No memory)

Extern is short name for external. Used when a particular file needs to access a variable from another file.

Scanned with CamScanner

Q. At same address create 2 files

extern1.c
extern2.c

extern1.c

```
#include<stdio.h>
extern int a;
int main()
{
printf("%d",a);
return 0;
}
```

Work inside code blocks only

extern2.c `int a=5;`

If in extern1 I write

```
extern int a;
extern int a;
```

} It is allowed

∴ multiple declarations are allowed but definitions are not.

If we delete extern2.C, and in extern1.c

```
#include<stdio.h>
int a=9;
int main(){
extern int a;
printf("%d",a);
return 0;
}
```

It will go to compiler & out of my code & you can find def

extern

but

ext

Compiler

Said i

thro

such

modifier

Ignoring

of acce

Ratio

Register

Keyword

Syntax se

Register

Variab

This is

Firstly

This is

the gi

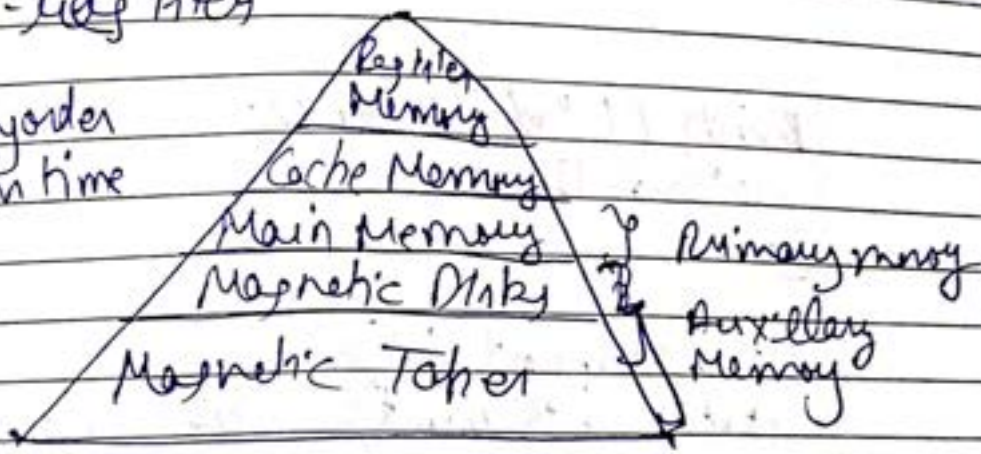
usually

need

extern int a=9; //allowed but here declared & defined both, so extern is redundant

Compiler believes whatever extern variable said is true & produce no error. Linker throws an error when he finds no such variable exist.

Modifier - register



Register Memory - Holds less content but fast to access
Magnetic Tape → more, slow.

Syntax register data-type var-name;

- Register keyword hints the compiler to store a variable in register memory.
- This is done because access time reduces greatly for most frequently referred variables.
- This is the choice of compiler whether it puts the given variable in register or not
- Usually compiler themselves do the necessary optimizations.

Scanned with CamScanner

Variable modifiers - Static

```
#include <stdio.h>
  ,  ,  <stdlib.h>
int increment();
int main(){
  int value;
  value = increment();
    value = ...;
    ... = ...();

printf("%d", value);
return 0;
}

int increment(){
  static int count;
    count+=1
    return count;
}
```

Ans = 3

Takeaways

1) Static variable remains in memory even if it is declared within a block on the other hand automatic variable is destroyed after the completion of function in which it was declared
2) Static variable if declared outside the scope of any function will act like global variable but only

Scanned with CamScanner

within the file in which it is declared.
5. You can only assign a constant literal (or value) to a static variable.

Constants in C?

What is constant?
→ As the name suggests - something that never change
→ Once defined can't be modified later in the code

Defining constants

#define const

Also called as Macro

ing #define

Syntax: #define NAME value

Job of preprocessor (not compiler) to replace NAME with value.

eg #include <stdio.h>
#define PI 3.141592
int main() {
printf("%.5f", PI);
return 0;
}

Scanned with CamScanner

Takeaways

1) Please don't add semicolon at
2) Choosing capital letters for end NAME is good practice.
3) We can use macros like functions. for eg

```
#include<stdio.h>
#define add(x,y) x+y
int main(){
printf("addition of two numbers: %d",
        add(4,3));
return 0;
}
```

4) We can write multiple lines using \

Eg-1

```
#include<stdio.h>
#define greater(x,y) if(x>y) \
        printf("%d is greater than %d",x,y); \
        else \
        printf("%d is lesser than %d",x,y);
int main(){
greater(5,6);
return 0;
}
```

Scanned with CamScanner

5) first expansion then evaluation

```
#include <stdio.h>
#define add(x,y) x+y
int main()
{
    printf("result of expression a*b+c is: %d", 5*add(4,3));
    return 0;
}
```

Ans = 23 not 5*7 = 35 but 5*4+3 = 20+3 = 23

This diff of function with macro.

6) Some predefined macros like __DATE__ __TIME__ can print current date & time

```
#include <stdio.h>
int main() {
    printf("Date: %s\n", __DATE__);
    printf("Time: %s\n", __TIME__);
    return 0;
}
```

Scanned with CamScanner

2) Using const

Syntax: const some_data_type some_variable

Note → we can't change const value

```
#include<stdio.h>
int main(){
const int var=57;
printf("%d", var);
return 0;
}
```

Question set-2

1. Output of

```
int main(){
int var=052;
printf("%d", var);
return 0;
}
```

a) 52
b) 56
c) 42
d) Ombiguous error

Ans (c):

052 → octal

0 in front of variable means octal (VVI)

So we are assigning octal and trying to print %d → decimal

so octal converted to decimal

8 | 80
5 | 2

5X8 + 2 = 42

Scanned with CamScanner

PAGE NO.: 287
DATE:

Q) What is %0d in place of %d in Integer

a) 52

Q) O/P of #include <stdio.h>
#define STRING "%s\n"
#define NESO "Welcome to Neso Academy!"
int main() {
printf(STRING, NESO);
return 0;
}

a) Compiler error b) "Welcome to Neso Academy!"
c) Garbage value d) Welcome to Neso Academy!

Ans (d)

Standard Input function - SCANF

→ Stands for Scan formatted String
→ Accept character, string & numeric data from user using standard I/P - keyboard
→ Scanf also use format specifiers like printf. For eg - %d to accept I/P of integer type
%c to accept I/P of character type
%s to accept a string
and go on - - -

Scanned with CamScanner

~~& - percent~~

& - ampersand

Eg

```
int var;
scanf("%d", &var);
```

→ Don't forget to add & in front of variable name

Why &?

→ While scanning the I/P, scanf needs to store that I/P data somewhere.
→ To store this I/P data, scanf needs to know the memory location of a variable.

& is also called as address-of operator

So &var => address of var

Simple prog

```
#include<stdio.h>
int main()
{ int a, b;
  printf("Enter the first number\n");
  scanf("%d", &a);
  printf("Enter the second number\n");
  scanf("%d", &b);
  printf("%d + %d = %d", a, b, a+b);
  return 0;
}
```

Scanned with CamScanner

Programming Questions (Set-2)

PAGE NO.: 289
DATE:

```
#include <stdio.h>
int main(){
  int var = 0x43ff;
  printf("%x", var);
}
```

43ff (0x & 0X both are same)

but if inside printf %x 43ff
%X 43FF

if %d, 17407 (hexa-decimal)

What is O/P of following C program

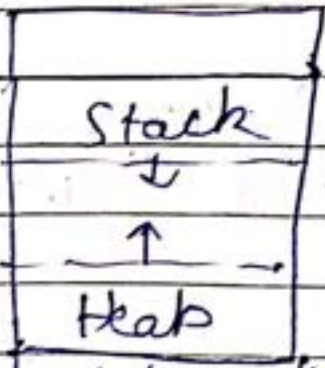
```
#include <stdio.h>
int main(){
  int var = 0x43ff
  static int i;
  while(i=27);
    printf("%d", i);
}
```

a) 27
b) 0
c) No o/p
d) None

```
int main(){
  static int i;
  printf("%d", i);
  return 0;
}
```

Scanned with CamScanner

In order to answer that question, let's see memory layout of C program



3 command line arguments & environment variables

Uninitialized global, static (both local & global), const global variables ← uninitialized data (bss)

Initialized data → Global, extern, static (both local & global), const global variables

Contains m/c code of the compiled program ← Text/code segment

Two memory segments.

1) Text/code segment
2) Data segment
   a) Initialized.
      i) Read only.
      ii) Read write
   b) Uninitialized (bss - Block started by symbol)
   c) Stack
   d) Heap.

Note - 1 If we write static int i;
static int i = 0;
both stored in bss.
Static always initialized (set 0)
If static int i = 24;
stored in data segment

Scanned with CamScanner

Now

static int i;
, , , i=27; Stored in data not bss
∴ i = 27 not 0

Now static int i;
static int i=27;
, , , , ;
↓
No changes

If static int i;
static int i=27;
static int i=14; // Compiler error
// Can't initialize static twice

If static int i=0
, , , i=27; // Compiler error

So answer = 27 a) but it's not right way

#include <stdio.h>
static int i; //127
static int i=27; //127
static int i; //127
int main() {
static int i; //0 (Simple static variable)
printf("%d", i); //0
return 0;
}

Ans (b)

Scanned with CamScanner

Let's explore more

→ #include <stdio.h>
static int i;
i=27;
i=45;
int main()
{
printf("%d", i);
return 0;
}
// Compiler error (Redef^n of i)

→ #include<stdio.h>
static int i;
static int i=27;
int main(){
i=45;
printf("%d", i); //45
return 0;
}

Conclusion → Can't use assignment for global
can , , , local

Motivation & Introduction to Operators in C

PAGE NO.:
DATE: 28

Types of operators in C

Name of operators	operators
Arithmetic operators	+, -, *, /, %
Increment/Decrement operators	++, --
Relational operators	==, !=, <=, >=, <, >
Logical operators	&&, ||, !
Bitwise operators	&, ^, |, ~, >>, <<
Assignment operators	=, +=, -=, *=, /= %=, <<=, >>=, &=, ^=
Other operators	?: (Ternary / Conditional), & (Ampersand / Address of), * (pointer), sizeof(), , (Comma)

Arithmetic operators

+, -, *, /, %

addition, subtraction, multiplication, division, modulus

All are binary operators ⇒ means two operands are required to perform operation. For eg: (A) + (B)

op1 op2

Scanned with CamScanner

Eg code

```
#include<stdio.h>
int main(){
int a,b;
printf("enter the first number\n");
scanf("%d\n",&a);
printf("Enter the second number\n");
scanf("%d\n",&b);

printf("a/b - %d\n",a/b);
printf("a%%b = %d\n",a%b);
```

double % printed (Imp)
#use %% before center will print %

Operators - precedence & associativity

Precedence	Operators	Associativity
↓ Highest	+, /, %	Left to right
lowest	+, -	Left to right

Note: Associativity is used only when two or more operators are of same precedence.

Eg: +, -

Same precedence : we use associativity

```
#include<stdio.h>
int main(){
int a=2,b=3,c=4,d=5;
printf("a*b/c - %d\n",a*b/c);
printf("a+b-c = %d\n",a+b-c);
printf("a+b+d-c%%a = %d\n",a+b+d-c%a);
```

PAGE NO. 95
DATE: / /

Increment & decrement operators in C

Increment operator is used to increment the value of a variable by one. Similarly decrement operator is used to decrement the value of a variable by one.

Increment.

```
int a=5
a++;
// a=6
```

Decrement

```
int a=5
a--;
// a=4
```

a++; is same as a=a+1;
a--; " " " a=a-1;

Both are unary operators

Pre-increment operator
++a;

Post-increment operator
a++;

Pre-decrement operator
--a;

Post-decrement operator
a--;

We cannot use rvalue before or after increment/decrement operator.

```
(a+b)++; // error
++(a+b); // error
```

error: lvalue required as increment operand

What are rvalues & lvalues?

Scanned with CamScanner

lvalue (left value): simply means an object that has an identifiable location in memory (i.e having an address)
- In any assignment statement lvalue must have the capability to hold the data
- lvalue must be a variable because they have the capability to store data
- lvalue cannot be a function, expression (like a+b) or a constant (like 3, 4 etc)

rvalue (right value): simply meaning an object that has no identifiable location in memory.
- Anything which is capable of returning a constant expression or value
- Expressions like a+b will return some constant value

For eg
a++ ✓ (a+b)++ ✗
a = a+1 (a+b) = (a+b)+1

↳ Compiler is expecting a variable as an increment operand but we are providing an expression (a+b) which does not have the capability to store data

Scanned with CamScanner

Diff b/w Pre & Post increment (a=5)

PAGE NO. 29
DATE:

x = ++a;

x	a
6	6

x = a++;

x	a
5	6

Q) What is o/p of ?

```
#include <stdio.h>
int main() {
  int a=4, b=3;
  printf("%d", a++b);
  return 0;
}
```

Ans) Let's Analyze

Token Generation

Lexical analysis is first phase in compilation
" analyzer (scanner) scans the whole program source program & when it finds the meaningful sequence of character (lexemes) then it converts it into a token

Token: lexemes mapped into token-name & attribute-value.

Eg: int -> <keyword, int>

It always matches the longest character sequence.

int a = 5;

lexical analyzer

int	a	=	5	;

starts from i-n-t (int) then space then a then = then 5 and so on.

If no space then from what it finds next char is invalid

a+++b

↓ Lexical analyzer

[a] [++] [+] [b]

∴ a++ is lvalue ∴ +b is rvalue

Now a++ +b = 4 3 (a = 5, b = 3)

Post increment / decrement in context of eqn:

→ First use the value in the eqn and then increment the value

Pre increment / decrement in context of equation:

→ First increment the value and then use in the eqn after completion of eqn.

So Ans → 7

Q2 What is O/P of :

```
#include <stdio.h>
int main() {
    int a = 4, b = 3;
    printf("%d", a++ +b);
    return 0;
}
```

Ans = 8

Q. o/p of a++++b

Ans [a] [++] [+] [+] [b]

↓

4 + 4 = 8 ✗ wrong error

but a++ + ++b no error.

Rel Relational operators

==, !=, <=, >=, < >

equal to, not equal to, less than equal to, greater than equal to, less than, greater than

→ used for comparing 2 values

→ All relational operators will return either true or false.

4==5 false

4!=5 True

Logical operators

&& , || , ! → NOT

AND OR

→ && and || are used to combine 2 conditions

&& - returns TRUE when all the conditions under consideration are true & returns FALSE when any one or more than one condition is false

|| - returns TRUE when one or more than one condition under consideration is true & returns FALSE when all conditions are false.

→ ! operator is used to complement the condition under consideration. ! - returns true when condition is false & returns false when condition is true.

Concept of short circuit in logical operators

Short circuit in case of && simply means if there is a condition anywhere in the expression that returns false, then the rest of the conditions after that will not be evaluated.

eg
```
#include<stdio.h>
int main(){
    int a=5, b=3;
    int inch;
    inch = (a<b) && (b++);
    printf("%d\n", inch);
    printf("%d", b);
    return 0;
}
```

if a<b make it 1

Short is a exp the will

eg #i i 1 1 y In & NOD + Bi a) I a ii) A c) #

Scanned with CamScanner

Short circuit in ||: simply means if there is a condition any where in the expression that returns True, then the rest of the conditions after that will not be evaluated.

```
eg #include<stdio.h>
   int main(){
   int a=5, b=3;
   int incr;
   incr = (a>b) || (b++);
   printf("%d\n", incr); //1
   printf("%d", b);      //3
   return 0;
   }
```

Introduction to bitwise operators (Logical operators)

&, |, ~, <<, >>, ^ (Ex-OR or XOR)

AND OR NOT left shift right shift

- Bitwise & operator
  - It takes 2 bits at a time & perform AND operation
  - AND (&) is binary operator. It takes 2 # & perform bitwise AND
  - Result of AND is 1 when both bits are 1

eg

```
  7  0111
& 4  0100
-------
  0  0100
```

7 & 4 = 4

Scanned with CamScanner

→ Bitwise OR(|) operator

a) It takes 2 bits at a time & perform OR
b) OR(|) is binary operator. It takes 2 number & perform bitwise OR
c) Result of OR is 0 when both bits are 0.

Eg
q → 0111
4 → 0100
0111
7|4=7

Bitwise NOT(~) operator

a) NOT is a unary operator
b) Its job is to complement each bit one by one
c) Result of NOT is 0 when bit is 1 & vice versa

q ~7 = ~0111
= 1000
~7=8

Diff. b/w Bitwise & logical (and in this eg)

```
#include <stdio.h>
int main() {
  char x=1, y=2;
// x=1 (0000 0001), y=2(0000 0010)
x & 2
if (x & y)          // 1 & 2 = 0 (0000 0000)
  printf("Result of x & y is 1");
if (x && y)         // 1 && 2 = TRUE && TRUE = TRUE = 1
  printf("Result of x && y is 1");
return 0;
}
```

Scanned with CamScanner

Left shift operator

First operand << Second operand
↓
whose bits get left shifted
Decides the # of places to shift the bits

Imp points
1. When bits are shifted left then trailing positions are filled with zeros.

```
#include <stdio.h>
int main(){
  char var = 3; //0000 0011
  printf("%d", var<<1);
  return 0;
}
```

Here var = 3
3 = 0000 0011
var<<1 0000 011_
↓
0000 0110 = 6

Left shifting is equivalent to multiplication by $2^{\text{right operand}}$

eg) var = 3
var<<1 O/P of [3 X 2^1]

2) var<<4 O/P of [3 X 2^4]

Scanned with CamScanner

Right Shift operator.

First operand >> Second operand

↓ ↓

whose bits get right shifted decides # of places to shift

Imp points

(1) When bit are shifted right then leading positions are filled with zeros.

```
#include <stdio.h>
int main(){
  char var=3;
  printf("%d", var>>1);
  return 0;
}
```

var>>1, var=3

3 = 0000 0011

0000 0001

0000 0001 = 1

(2) Right shifting is equivalent to division by right operand

Eg var=3

var>>1 // 1 $\left(\frac{3}{2^1}\right)$

var=32

var>>4 // 2 $\left(\frac{32}{2^4}\right)$

Scanned with CamScanner

Bitwise XOR operator

PAGE NO: 3 of
DATE:

Exclusive OR / or simply OR → Either A is 1, B is 1 or Both are 1, then the o/p is 1
→ Including Both

Exclusive OR → Either A is 1 or B is 1 then the o/p is 1 but when both A & B are 1 then o/p is 0
→ Excluding BOTH

→ Bitwise XOR (^) is binary operator. It takes 2 number & perform bitwise XOR.
→ Result of XOR is 1 when 2 bits are diff otherwise the result is 0

Eg 7 0111
^ 4 0100
0011 ∴ 7 ^ 4 = 3

Assignment operator
→ Value to a variable can be assigned using assignment operator.
→ Requires 2-values - L-value & R-value
→ This operator copies R-value to L-value.

Eg var = 5;
L-value ↓ R-value
Assignment operator

Scanned with CamScanner

Shorthand assignment operators

Note - Assignment operator is having least precedence among all operators

+= first addition then assignment
-= // sub " " "
*= " mult " " "
/= " division " " "

a+=1 is same as a=a+1;

other %= first modulus then assign
<<= first bitwise left shift " "
>>=
&=
|=
^=

14/9/20 Introduction to Conditional operator

? : Remember it's ternary (3 operands)

why required?

```
char result;
int marks;
if(marks>33)
  result='p';
else
  result='f';
```

Better `result=(marks>33)?'p':'f';`

Scanned with CamScanner

PAGE NO.: 307
DATE: / /

Marks > 33 is a boolean expression.
it will return either TRUE or FALSE

result = (mark > 33) ? 'P' : 'F';

TRUE → 'P'
FALSE → 'F'

expression is r-value
result is l-value

Quick facts
→ conditional operator is the only ternary operator in C.

```
#include <stdio.h>
int main(){
int var=75;
int var2=56;
int num;
num=sizeof(var)?(var2>23?((var==75)?
'A':0):0):0);
printf("%d\n",num)
}
```

Ans - 65

Scanned with CamScanner

Comma operator

(1) Comma operator can be used as a "separator".

Eg `int a=3, b=4, c=8;` = `int a=3; int b=4; int c=8;`

(2) Comma operator can be used as an operator.

Eg
```
int a=(3,4,8);
printf("%d", a);
```

Comma operator returns the rightmost operand in the expression & it simply evaluates the rest of the operands & finally rejects them.

Eg `int var=(printf("%s\n", "Hello"), 5);`

"Hello" evaluated then rejected, so Hello will not be printed. var = 5

Illy `In printf("%d", var);` same function

O/P = HELLO!
O/P = 5

Scanned with CamScanner

3) Comma operator is having least precedence among all the operators available in C language

```
eg1 int a;          ==  int a;
    a = 3, 4, 8;    ==  (a = 3), 4, 8;
    printf("%d", a);    printf("%d", a);
    //3
```

```
Eg2. int a = 3, 4, 8;   //Here comma is behaving like a separator
     printf("%d", a);
```

So this is int a=3; int 4; int 8;
error (not possible)

```
Eg3 int a;
    a = (3, 4, 8);
    printf("%d", a); //8
    //1
    int a = (3, 4, 8);
    ↑↑
```

Bracket has highest precedence than any other operator

comma operator not separator

```
Eg4 #include <stdio.h>
    int main(){
    int var;
    int num;
    num = (var = 15, var += 35);
    printf("%d\n", var);
    }//50
```

Scanned with CamScanner

Precedence of operators

Precedence of operators came into picture when in an expression we need to decide which operator will be evaluated first. Operator with higher precedence will be evaluated first.

Associativity of operator

→ Come into picture when precedence of operators are same and we need to decide which operator will be evaluated first.

either 1) Left to Right OR
2) Right to left

Precedence & Associativity Table (Memorize this)

Category	Operator	Associativity
Parenthesis	() [] -> . ++ -- (++ and -- are postfix)	L→R
Unary	! ~ ++ -- + - * & sizeof (++ and -- are prefix)	R→L
Multiplicative	* / %	L→R
Additive	+ -	L→R
Bitwise Shift	<< >>	L→R
Relational	< <= > >=	L→R
Equality	== !=	L→R
Bitwise AND	&	L→R
Bitwise XOR	^	L→R
Bitwise OR	|	L→R
Logical AND	&&	L→R
Logical OR	||	L→R
Conditional	? :	R→L
Assignment	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=	R→L

Scanned with CamScanner

1) comma (,) L→R

Let's talk about all 1 by 1

PAGE NO.: 34/19
DATE:

1) ( ) - paranthesis in function calls
eg: int var = fun( );
- operator is having less precedence as compared to ( )
- ( ) belongs to fun & fun will be treated as a function.

2) Member access operators (-> .)
They are used to access member of structures we'll talk about structures later

3) Postfix Increment/Decrement (++, --)
- Precedence of Post... > prefix in this
- Associativity of ++ is also diff from postfix L→R, Prefix R→L.

4) Unary + - ! ~ ++ -- (type) sizeof
They are diff from Arithmetic operators.
eg. var = +34, var = -24

5) Unary (type) is cast operator (later).

Scanned with CamScanner

Imp facts

→ Associativity can only help if there are 2 or more operators of same precedence & not when there is just one operator

Eg →
```
int main()
{ int a;
a = fun1() + fun2();
printf("%d", a);
return 0;
}
```

```
int fun1(){
printf("Nero");
return 1;
}
```

```
int fun2(){
printf("Academy");
return 1;
}
```

Which is called first? fun1() or fun2()

→ It is not defined whether fun1() will be called first or whether fun2() will be called. Behaviour is undefined & o/p is compiler dependent.

Note: Associativity will not come into picture as we have just one operator & which function will be called first is undefined. Associativity will only work when we have more than one operator of same precedence

So o/p : NeroAcademy

Or

AcademyNero

Scanned with CamScanner

2) Same precedence operators have same associativity.

```
#include<stdio.h>
int main(){
  int a=10, b=20, c=30, d=40;
  if(a<=b == d>c)
    printf("TRUE");
  else
    printf("FALSE");
}  //TRUE
```

Q) What is O/P of

```
#include<stdio.h>
int main(){
int i=5;
int var=sizeof(i++);
printf("%d %d", i, var);
}
```

a) 5 4
b) 6 4
c) 5 8
d) Compiler error

Ans) Assume size of integer is 4 bytes

R.S c4 is not correct, why? Let's see— standard is the language specification which is adopted by all C compilers across the globe.

Acc to C99 standard: The size of operator yields the size (in bytes) of its operand, which may be an expression or a parenthesized name of a type. The size is determined from the type of operand. If the type of operand is a variable length array type, then the operand is evaluated otherwise, the operand is not evaluated and the result is an integer constant.

Scanned with CamScanner

C99 is older version of C standard adopted in 1999.

C11 is latest revised version of C adopted in 2011.

Q. O/P of
```
int a=1;
int b=1;
int c = ++a || b++;
int d = b-- && --a;
printf("%d %d %d %d", d, c, b, a);
```
a) 1111
b) 0100
c) 1001
d) 1101

Ans
`c = ++a || b++;` , c=1
2 Not evaluated

`d = b-- && --a;` , d=1
1 1

a = 0 = c = d = 1
b = 0 after decrement.

d) ans

Q.
```
#include <stdio.h>
int var = 5;
int main() {
int var = var;
printf("%d", var);
}
```
a) 5 b) Compile error
c) Garbage value
d) None

Scanned with CamScanner

If-else, Nested-if, Elseif

PAGE NO.: 31
DATE:

FAQs?

Q) Is it necessary to put else part?
Ans) Answer is No & YES at same time.
Let's see why

Eg 1
```
int n=4
if (n!=4)
printf("n is not 4");
```
No O/P

Eg 2
```
int n=4
if (n!=4)
printf("n is
else
printf("n is 4");
```
n is 4

Switch - Replacement to long if else ladder

Eg
```
int x=2;
if(x==1)
  printf("x is 1");
else if (x==2)
  printf("x is 2");
else if (x==3)
  printf("x is 3");
else
printf("x is other than 1, 2 & 3");
```

=

```
int x=2;
switch(x) {
case 1: printf("x is 1");
  break;
case 2: printf("x is 2")
  break;
case 3: printf("x is 3")
  break;
default: printf("x is a
number other than 1,
2 & 3");
  break;
```

Scanned with CamScanner

- Default is optional

- Suppose case1 is satisfied. and there is no break after printf then subsequent expression will also get evaluated until we reach next break.

facts related to switch

(1) You are not allowed to add duplicate case

(2) only those expressions are allowed in switch which results in an integral constant value.

Allowed

```
int main(){
int a=1,b=2,c=3;
switch(a+b*c){
case1:printf("choice1");
   break;
case2: printf("choice2");
   break;
default: printf("default");
   break;
}
}
```

Not allowed

```
int main(){
float a=1.5,b=20,c=30;
switch(a+b*c)
{
case1:
}
}
```

(3) float value is not allowed as a constant value in case label. Only integer constants / constant expressions are allowed in case label.

Scanned with CamScanner

Variable expressions are not allowed in case labels. Although macros are allowed in case.

```
int main(){
int x=2; y=2, z=23;
switch(x){
Case y: printf("Number is 2");
    break;
case z: printf("Number is 23");
    break;
default: break;
}
```

```
#include<stdio.h>
#define y 2
#define z 23
int main(){ int x=2;
switch(x){
 case y: printf("Number is 2");
    break;
 Case z: printf("Number is 23");
    break;
}
}
```

Default can be placed anywhere inside switch. It will still get evaluated if no match is found.

Scanned with CamScanner

Importance of loops

1) WHILE loop

Syntax:
while (expression)
{
statement 1;
statement 2;
!
!
}

Eg:
int i=3;
while (i>0){
printf(i);
i--;
}

2) FOR loop
for (initialization ; condition ; increment/decrement){
statements;
}

Eg → int i; for (i=3; i>0; i--)
{
printf (i); //321
}

Diff b/w while & do-while

1) While
int i=0;
while (i>0){
printf("%d", i);
i--;
}
// NO O/P

do-while
int i=0
do{
printf("%d", i);
i--;
} while (i>0);
// 0

2) ; after do-while kw. not after while

When should I prefer do-while over while?

Eg- Write a program which allows user to enter an integer until he/she enters a value zero.

```
int n;
printf("Enter an integer\n");
scanf("%d", &n)

while (n != 0) {
printf("Enter an integer\n");
  scanf("%d", &n);
}
printf("You are out of the
  loop")
```

```
int n;
do {
printf("Enter an integer\n");
  scanf("%d", &n);
} while (n != 0);
printf("You are out of
  the loop").
```

Loop control statement : Break & Continue

Break: Used to terminate the loop.

Eg- WAP to allow user to enter integers until he/she enters a -ve number or zero.

```
#include<stdio.h>
int main(){
  int n;
  printf("Enter a number\n");
  scanf("%d", &n);
while (n != 0) {
  if (n<0) break;
  printf("Enter a number\n"); scanf("%d", &n);
}
```

Scanned with CamScanner

Continue

- Similar to break but instead of terminating the loop, it forces to execute the next iteration of the loop.

Eg - WAP to print all odd nos from 1 to 20.

```
#include<stdio.h>
int main(){
int i, n=2;

for(i=1; i<=20; i++)
  if(i==n){
    n+=2;
    continue;
  }
printf("%d ", i);
```

Q How many times will "Hello, World" be printed

```
#include<stdio.h>
int main(){
int i=1024;
for(; i>>=-1)
printf("Hello, World");
return 0;
}
```

a) 10
b) 11
c) Infinite
d) None

Q. 1 O/P of
```
#include <stdio.h>
int main() {
int i;
for (i=0; i<=20; i++) {
switch(i) {
case 0: i+=5;
case 1: i+=2;
case 5: i+=5;
default: i+=4;
}
printf("%d ", i);
}
```
a) 5 10 15 20
b) 7 12 17 22
c) Compile error
d) 16 21
Hint: no break

Q.
```
#include <stdio.h>
int main() {
int i=0;
for(printf("one\n"); i<3 && printf(" "); i++)
printf("Hi\n");
```
One : condition false also

Q. O/P of
```
#include <stdio.h>
int main() {
unsigned int i=50;
while(i++ != 0);
printf("%d", i);
return 0;
}
```
a) 0
b) 1
c) Infinite Loop
d) Compile error

Scanned with CamScanner

for 5 semicolon at end of while so print not part of while

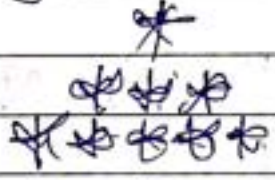
Ans(b) i

→ Range of unsigned int (4 byte) 0 to 4294...
→ When i reaches 4294..., then because of i++, it comes back again to 0
→ As 0 not equal to 0 is false ∴ we come outside of while loop
→ After we come out of loop, i will contain 1 because of post increment operator ∴, 1 will be printed on the screen.

Special problem: Pyramid of stars

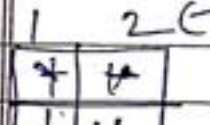
Console → How many rows you want in your pyramid?

3

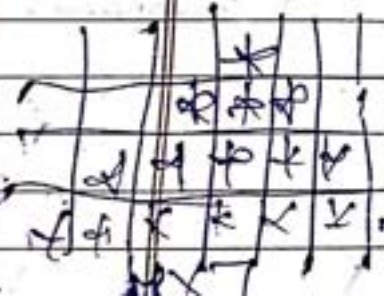


Let's solve,

Matrices




```
for(i=1; i<=2; i++)
{ for(j=1; j<=2; j++)
  { printf("*"); }
}
```





3 × 5

Scanned with CamScanner

3 rows -> 5 colums 5 rows -> 9 colums PAGE NO. 323
4 row -> 7 colums 6 rows -> 11 colums DATE:
n rows -> 2n-1 colums

Soln is
```
for(i=1; i<=n; i++)
{ for(j=1; j<=2*n-1; j++)
  { if(j>=n-(i-1) && j<=n+(i-1))
      printf("*");
    else
      printf(" ");
  }
  printf("\n");
}
```

Palindrome: A number or a word or a phrase if read backwards gives the same number or a word or a phrase as it is being forward
eg 121, racecar, 29542, etc.
Here we are doing for number only.
Logic: Simply reverse the # and check if reverse = actual

Code
```
#include<stdio.h>
int main(){
  int n, result=0, q, rem;
  printf("Please enter the number:");
  scanf("%d", &n);
  q=n;
  while(q!=0){
    rem = q%10; result = result*10+rem;
    q=q/10;}
  if(result==n) printf("It's a palindrome");
  else printf("Not");}
```

Scanned with CamScanner

Armstrong number: An armstrong number of order n is a number in which each digit when multiplied by itself n number of times and finally added together, result the same number.

Eg 371 , $3^3+7^3+1^3=371$

Step #1 → find #1 of digits.

Step #2 ⇒
```
count = 0;
while (q != 0){
    q = q/10;
    count++;
}
```

eg 371/10 = 37 count 1
37/10 = 3 count 2
3/10 = 0 count 3

Step #2: Multiply each digit n (here 3) times and add them

```
cnt = count, result = 0
while (q != 0){
rem = q%10;
while (cnt != 0){
mult = mult * rem;
cnt--;
result = result + mult;
cnt = count;
q = q/10;
mult = 1;
```

1) 371%10 = 1
mult = 1
result = 1

2) 37%10 = 7
mult = 7*7*7 = 343
result = 1+343
= 344

3) 3%10 = 3
mult = 3*3*3 = 27
result = 344+27 = 371

4) 0%10 = 0

Step 3
Check if result = actual #
if (result == number) printf("YES");
else printf("NO");

Strong number: # in which sum of factorial of individual digits of a # is equal to original #.
Eg 145 = 1! + 4! + 5! = 1 + 24 + 120 = 145

Q. Write a program to find strong #?

Step #1 → Calculate factorial of each digit of a # and add them
q = n, fact = 1, result = 0;
while (q != 0) {
rem = q % 10;
for (int i = 1; i <= rem; i++) {
fact = fact * i; }
result = result + fact;
fact = 1;
q = q / 10; }

Step #2 - Check result with actual #

WAP if # is prime (let's say 23)
How to check → Step 1 → Take square root of 23. which is nearly 4.7958 = 5
Step 2 - Divide 23 by numbers greater than 2 & less than or equal to 5. If it is divisible by any of those numbers then it is not prime else it is prime.
Eg 23 is not divisible by any of 2 to 5 so it is prime.

Concept: In order to find whether a number is prime or not, We first need to calculate the square root of that number & then we divide that # by numbers less than or equal to the square root of that number. If it is divisible by any of the numbers then we can say that number is not prime else it is a prime number.

```
#include <stdio.h>
#include <math.h>
int main(){
  int x;
  int i=1, val1, val2, count=0;
  printf("Please enter a # : ");
    scanf("%d", &x);
  val1 = ceil(sqrt(x));
  val2 = x;

for(i=2; i<=val1; i++){
  if(val2%i==0)
    count=1;
}
if(count==0 && val2!=1 || val2==2 || val2==3)
  printf("%d is a prime #", val2);
else
  printf("%d is not a prime #", val2);
return 0;
}
```

Scanned with CamScanner

→ sqrt means square root
→ available in math.h library
Syntax: double sqrt(double x)

→ ceil function return a smaller integer greater than or equal to x
→ available in math.h library
Syntax double ceil(double x)

Eg: sqrt(2) = 1.414
ceil(1.414) = 2

Q. Add 2 no without using + operator.

Ans Idea is to use increment & decrement operators

x=3, y=4
Then: // x++; y--;
// repeat 1 while y=0

1: x=4, y=3
2 x=5, y=2
3 x=6, y=1
4 x=7, y=0
Ans

```
#include<stdio.h>
int main(){
  int x, y;
  printf("Enter the 2 Numbers you want to add:\n");
  scanf("%d %d", &x, &y);
  while(y != 0){
    x++;
    y--;
  }
  printf("Sum of 2 values is: %d\n", x);
}
```

Scanned with CamScanner

① Addition without + operator (-ve & +ve)

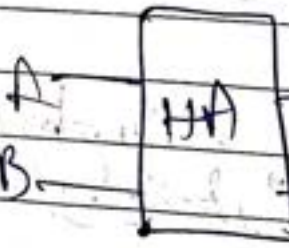
Ans:
```
#include <stdio.h>
int main() {
int x, y;
printf("Enter 2 numbers you want to add");
scanf("%d %d", &x, &y);
if (y > 0) {
while (y != 0) {
x++; y--; }
}else if (y < 0) {
while (y != 0) {
x--; y++; }
}
printf("Sum of 2 values is %d", x);
}
```

eg X=2 x--=1 Ans
Y=-1 y++=0

② Program to add 2 #s without using '+'

→ Idea: Use Half adder logic

Half adder Basics



A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$S = A \text{ xor } B$
$C = A \cdot B$
or A and B

Scanned with CamScanner

while (b != 0) {
sum = a ^ b;
carry = (a & b) << 1;
a = sum;
b = carry;
}

a=5 → 0101
b=10 → 1010
15 → 1111
sum →
carry 0000

7 → 0111
2 → 0010
1001

Iteration 1: Sum 0101 €
Carry 0010 (but we are adding carry 3rd bit)

So carry << 1
So sum 0101
Carry 0100

Iteration 2 Sum 0101
Carry 0100
Error 0000

carry → 0100 << 1 → 1000

Iteration 3 0001
1000
1001 → Ans

7 → 0111
7 → 0111
14 1110

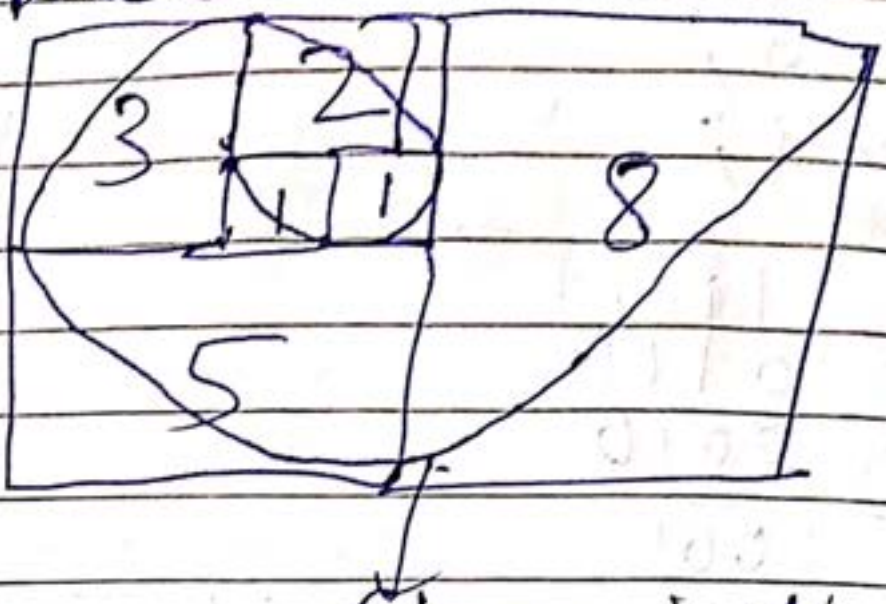
Iteration 1: Sum 0000
Carry 0111
→ 1110

Iteration 2
Sum 1110
Carry 0000 Stop

Scanned with CamScanner

Q. Print fibonacci series upto n number of terms.

fibonacci spiral.



fibonacci spiral

```
#include<stdio.h>
int main(){
  int a, b, result, n, i;
  printf("Enter the # of term: ");
  scanf("%d", &n);
    a=0;
    b=1;
for (i=1; i<=n; i++){
    printf("%d", a);
    result = a+b;
    a=b;
    b= result;
}
```

Scanned with CamScanner

What is Floyd's triangle?
Right ld Δular array of natural numbers
It is filled by consecutive natural numbers
forsecutively starting with 1 in the top
left corner

Ex- Floyd's Δ with 5 rows

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Print floyd's Δ of n rows.

```
#include<stdio.h>
int main() {
int i, j, rows, n=1;
printf("Enter the # of rows: ");
scanf("%d", &rows);
for(i=1; i<=rows; i++) {
for(j=1; j<=i; j++) {
printf("%d ", n);
n++;
}
printf("\n"); }
return 0;
}
```

Scanned with CamScanner

Binary to decimal

```
#include <stdio.h>
int main(){
  int decimal=0, binary, base=1, rem, num;
  printf("Enter the binary number: ");
  scanf("%d", &binary);
  num=binary;
  while(binary!=0){
  rem = binary % 10;
  decimal = decimal + rem*base;
  binary = binary/10;
  base = base*2; }

printf("Decimal equivalent of binary
  %d is: %d", num, decimal);
  return 0;
```

Power of an integer

eg $a^b = a \times a \times a \ldots$ b times

eg $2^3 = 2 \times 2 \times 2$

$2^{-3} = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2}$

```
#include <stdio.h>
int main(){
int base, exponent, exp, power=1;
double power1 = 1.0;
printf("enter the base: ");
scanf("%d", &base);
printf("enter the exponent: ");
scanf("%d", &exponent);
```

Scanned with CamScanner

expo - exponent;
if (exponent > 0) {
while (exponent != 0) {
power = power * base;
exponent --;
}
printf("%d to the power of %d is %d", base, expo, power);
} else {
while (exponent != 0) {
power = power * (1.0/base);
exponent ++;
}
printf("%d to the power of %d is %d", base, expo, power);
}

Leap year

Any year is leap year if it is exactly divisible by 4 except centurial year. But Centurial year are leap year if they are exactly divisible by 400

3 steps
1) If a year is divisible by 400 it is leap year
eg - 2000
2) If a year is not divisible by 400 but 100 then not a leap year, eg - 1900
3) If a year is not divisible by 400 & not by 100 but by 4, it is leap yea. eg - 2012

Scanned with CamScanner

```
#include <stdio.h>
int main()
{
    int year;
    printf("Enter the year: ");
    scanf("%d", &year);
    if (year % 400 == 0)
        printf("%d is a leap year", year);
    else if (year % 100 == 0)
        printf("%d is not a leap year", year);
    else if (year % 4 == 0)
        printf("%d is a leap year", year);
    else
        printf("%d is not a leap year", year);
    return 0;
}
```

Q- Perfect number

→ Perfect no. is a +ve integer that is equal to the sum of all its proper +ve divisors, excluding the number itself.

For eg: 6 is a perfect number

Because the proper +ve divisors of 6 are 1, 2 & 3 (excluding 6) and sum of all its +ve divisors i.e $1+2+3=6$

Scanned with CamScanner

Introduction to functions

PAGE NO. 3
DATE

Def:- Function is basically a set of statements that takes inputs, does some computation & produces output.

Syntax: Return type function name (set of inputs);

Why functions?

1) Reusability: Once defined, can be used over & over again

2) Abstraction: If you are just using the function in your program then you don't have to worry about how it works inside

Eg - Scanf function.

```
#include <stdio.h>
int areaOfRect (int length, int breadth){
  int area;
  area = length * breadth;
  return area;
}
int main(){
  int l=10, b=5;
  int area = areaOfRect(l,b);
  Printf("%d\n", area);
}
```

Scanned with CamScanner

Function declaration in C

Function declaration (also called function prototype) means declaring the properties of a function to the compiler. for eg: int fun(int, char);

Properties 1) Name of function: fun
2) Return type of function: int
3) Number of parameters: 2
4) Type of parameter 1: int
5) " " " 2: char

Imp- Fun prototype always end with ;

→ It is not necessary to put the name of parameters in function prototype
eg → int fun(int var1, char var2); ← Not necessary

Eg
```
#include <stdio.h>
char fun();
int main(){
char c = fun();
printf("character is: %c", c);
}
char fun(){
  return 'a';
}
```

→ It is not always necessary but it is preferred to declare the function before using it

PAGE NO.: 337
DATE: / /

```
#include <stdio.h>
char fun() {
    return 'a';
}
int main() {
    char c = fun();
    printf("Character is: %c", c);
}
```

What happens if we call function before defining/declaring it.

Error (implicit declaration of function 'fun')

Function def^ in C

Function def^ consists of block of code which is capable of performing some specific task.

Eg:-

```
int add(int a, int b) {
    int sum;
    sum = a+b;
    return sum;
}
```

Code

```
#include <stdio.h>
int add(int, int);   → no need of variable
int main() { int m=20, n=30, sum;
sum = add(m,n);   → Imp no need of storage
printf("sum is %d", sum); }
int add(int a, int b) {
return(a+b);
```

Scanned with CamScanner

In declaration it is imp to mention both name & data type & return type.

Diff b/w an argument & a parameter?

Parameter: is available in the declaration & definition of the function

Argument: is the actual value of the parameter that gets passed to the function.

Note: Parameter is also called as formal parameter & Argument is also called as Actual parameter.

Eg
```
int add(int, int);
int main(){
  int m=20, n=30, sum;
  sum=add(m,n);
  printf("sum is %d", sum); }
int add(int a, int b){
  return(a+b);
}
```
Arguments or actual parameters

Parameters or formal parameters

Call by value - Here values of actual parameters will be copied to formal parameters & these 2 different parameters store values in different locations.

```
int x=10, y=20;
fun(x,y);
int fun(int x, int y){
  x=20
  y=10;
```

x y | 10 20 | 20 10

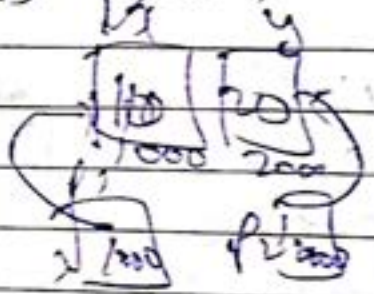
when we
printf("x=%d, y=%d", x, y);
o/p x=10, y=20

∴ x, y of fun are local to function & changes in those x, y won't change actual values of x & y.

Call by reference. Here both actual & formal parameters refers to same memory location. Therefore any changes made to the formal parameters will get reflected to actual. Here instead of passing values as parameters

```
int x=10, y=20;
fun(&x, &y);
```

```
int fun(int *p1, int *p2)
{ *p1 = 20;
  *p2 = 10;
}
```



printf("x=%d, y=%d", x, y);
o/p x=20, y=10

```
int fun(int num){
int count=0;
while(num){
count++; num>>=1; }
return count; }
```

1 1 0 1 1 0 0 1 1
2^8 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

value of fun(459) is 5

Date 2015 Sept

Q → O/P of

```
int fun() {
static int num = 1;
return num--;
}
int main() {
for (fun(); fun(); fun())
printf("%d", fun());
return 0;
}
```

a) Infinite loop
b) 3 1 0 -1
c) 4 1 1 8 5 2
d) 15 12 8 5 2

Ans → for (16; 15; fun(3))
printf("%d", 4);
fun()

15/9/20

Static functions in C

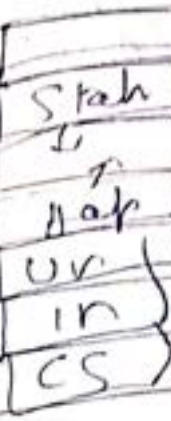
→ In C functions are global by default
→ This means if we want to access the function outside from the file where it is declared, we can do it easily.
→ Now if we want to restrict this access, then we make our function static by simply putting a keyword static in front of the function

Imp Takeaways
→ Static functions are restricted to the files where they are declared
→ Reuse of the same function name in other files is possible

Scanned with CamScanner

Static & Dynamic Scoping

Preliminaries



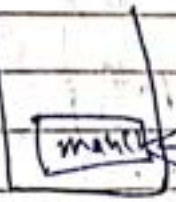
→ Stack is a container (2D) memory segment which holds some data
→ Data is retrieved in Last In First Out (LIFO)
→ Two operations: push & pop



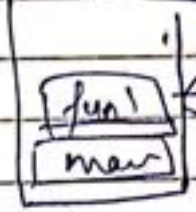
STACK or CALL STACK = Call stack

Stack means whenever you call function it gets stored in stack

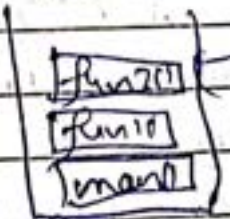
a) main() { fun }



b) main(){ fun1(); }



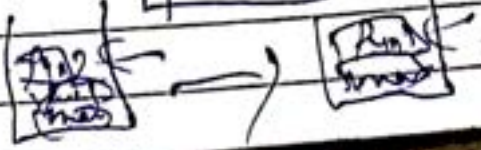
c) fun1(){ fun2(); }



d) fun2(){ fun3(); }



e) fun3(){ return; }

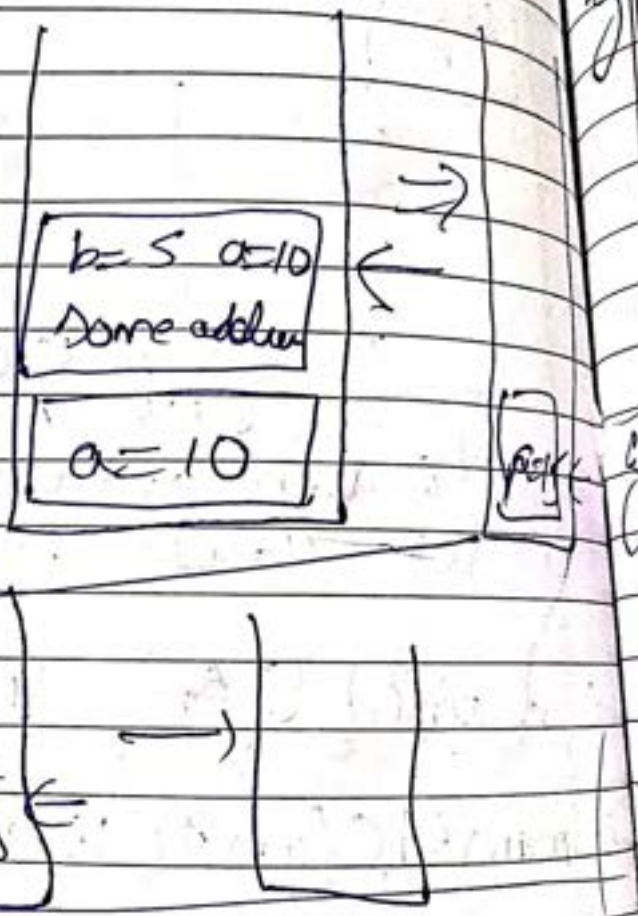


Scanned with CamScanner

In reality (not function) activation record of function is stored inside stack.

Activation Record (□) - is a portion of the stack which is generally unpaged.
1) Locals of the callee
2) Return address to the caller
3) Parameters of the callee

```
eg int main() {
   int a=10;
   a=fun1(a);
   printf("%d", a);
}
int fun1(int a){
   int b=5;
   b=b+a;
   return b;
}
```



Why scoping?
Scoping is necessary if you want to reuse variable names.

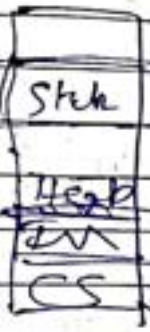
eg int fun1(){ int a=20;}
int fun2(){ int a=20;}

What is Static Scoping?

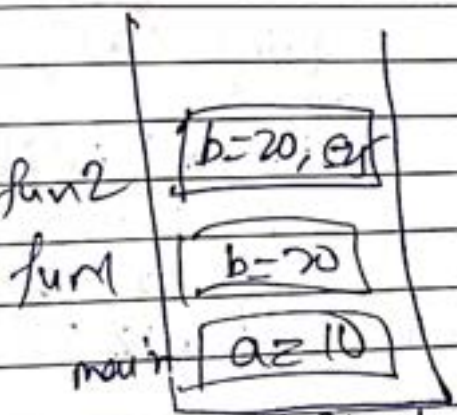
PAGE NO.
DATE: 1/1/20

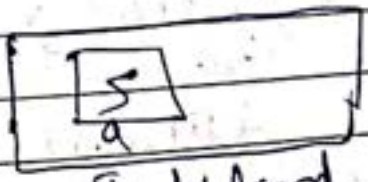
In static scoping or lexical scoping, definition of a variable is resolved by searching it's containing block or function. If that fails, then searching the outer containing block & so on.

```
int a=10, b=20
int fun(){
int a=5;
int c;
c=b/a;
printf("%d", c); //4
}
```




```
int fun1(int);
int fun2(int);
int a=5;
int main(){
int a=10;
a=fun1(a);
printf("%d", a);
}
int fun1(int b){
b=b+10;
b=fun2(b);
return b;
}
int fun2(int b){
int c;
c=a+b;
return c;
}
```





SOP = 25

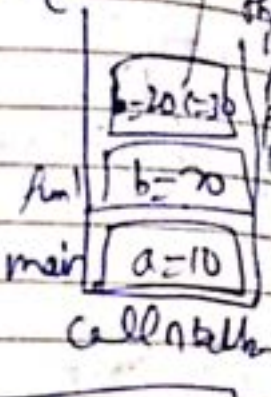
Scanned with CamScanner

Dynamic Scoping

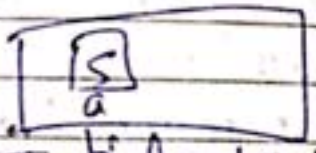
→ In dynamic scoping, def^n of variable is resolved by searching the containing block & if not found, then searching its calling function & if still not found then the function which called that calling function will be searched and so on

Eg-1 Source

```
int fun1(int);
int fun2(int);
int a=5;
int main(){
int a=10;
a= fun1(a);
printf("%d", a);
}
int fun1(int b){
b=b+10;
b=fun2(b);
return b;
}
int fun2(int b){
int c;
c=a+b;
return c;
}
```



Call stack



Static data segment

O/P = 30

Imp Takeaways

1) In most of the programming languages static scoping is followed instead of dynamic scoping
2) Languages including Algol, Pascal, C, Haskell, Scheme, etc. are statically scoped
3) Some languages including SNOBOL, APL, LISP etc. are dynamically scoped
4) As C follows static scoping, it is not possible to use nonlocal in C

→ Perl is a programming language which supports both static as well as dynamic scoping.

Eg $x = 50;
sub fun2 {
return $x;
}
sub fun1 {
local $x = 10;
my $y = fun2();
return $y;
}
print fun1();

→ local allows dynamic scoping
→ my allows static scoping
→ local keyword makes your variable visible up the call stack too.
→ my keyword makes your variable visible only when it's declared.

Ans = 10

Eg: In line (1) we replace local by my
Ans = 50

Consider the program below in a hypothetical programming language which allows global variables & a choice of static or dynamic scoping.

int i;
program main() {
i = 10;
call f();
}
procedure f() {
int i = 20;
call g();
}
procedure g() {
print i;
}

Let x & y be value printed under static & dynamic scoping then

a) x = 10, y = 10
b) x = 20, y = 10
c) x = 10, y = 20
d) x = 20, y = 20

Scanned with CamScanner

Basics of recursion

Defn-
Recursion is a process in which a function calls itself directly or indirectly

eg- int fun(){
    ---
    fun();
}

Program in fun(int n){
    if(n==1)
        return 1;
    else
        return 1+fun(n-1);
}
int main(){
    int n=3;
    printf("%d", fun(n)); // 3
    return 0;
}

You can see by call stack also

How to write a recursive program?
1) Divide the problem into smaller subproblems
2) Specify the base condition to stop recursion

Problem: Calculate factorial of a#

Basic structure
Fact(){
    if(){ } - Base Case
}
else { } Recursive procedure

Scanned with CamScanner

```
#include<stdio.h>
int fact(int n){
  if(n==1)
    return 1;
  else
    return n* fact(n-1);
}
int main(){
int n;
printf("Enter the number: ");
scanf("%d", &n);
printf("Factorial of a number %d is %d", n,
                                fact(n));
}
```

-1 Types of recursion
1) Direct recursion
2) Indirect "
3) Tail "
4) Non-tail "

1) Direct recursion - A function is called direct recursive if it calls the same function again

Structure of Direct recursion

```
fun(){
  //some code
  fun()
  //some code
}
```

Q Factorial

Scanned with CamScanner

2) Indirect recursion - A function is called indirect recursive if it calls another function and then fun2 calls fun directly or indirectly.

Structure

```
fun1(){
  // some code
  fun2();
  // some code
}
```

```
fun2(){
  // some code
  fun1();
  // some code
}
```

Program: WAP to print number from 1 to 10 in such a way that when number is odd add 1 & when number is even subtract 1.

O/P: 2 1 4 3 6 5 8 7 10 9

1+1 2-1 3+1 4-1 and so on

```
void odd();
void even();
int n=1;
void odd(){
  if(n<=10){
    printf("%d", n+1);
    n++;
    even();
  }
  return;
}
```

```
void even(){
  if(n<=10){
    printf("%d", n-1);
    n++;
    odd();
  }
  return;
}
int main(){
  odd();
}
```

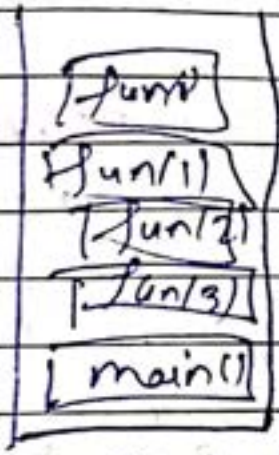
PAGE NO.: 349
DATE: 3/1/19

Tail recursion

A recursive function is said to be tail recursive if the recursive call is the last thing done by the function. There is no need to keep record of previous state

```
void fun(int n){
  if(n==0)
    return;
  else
    printf("%d", n);
  return fun(n-1);
}
int main(){
  fun(3);
  return 0;
}
```

o/p: 3 2 1



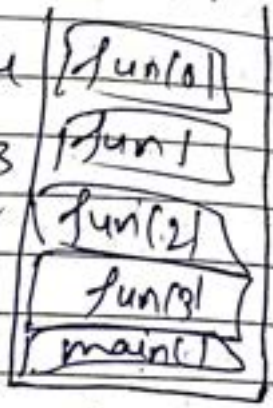
Call stack (no updates no need to keep previous record)

In most cases when we return back from top of stack there is need of evaluation (to keep record), but here there is no need to backtrack.

Hence, we can say that tail recursive functions do not utilize the extra space of stack Properly

4) Non-tail recursive → A recursive function is said to be non-tail recursive if the recursive call is not the last thing done by the function. After returning back, there is some something left to evaluate.

eg
```
void fun(int n){
  if(n==0)
    return;
  fun(n-1);
  printf("%d", n);
}
int main(){
  fun(3);
  return 0;
}
```

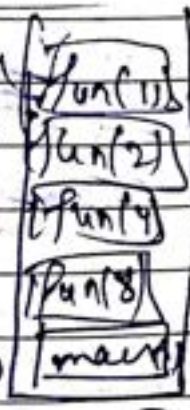


While we return, at step 6, we need to evaluate printf
i/p step 7, 8

So O/P 1 2 3 (Just opp" to tail recursive)

one more eg (Non tail)

```
int fun(int n){
  if(n==1) return 0;
  else return 1 + fun(n/2); }
int main(){
  printf("%d", fun(8));
  return 0;
}
```



Ans 3

Non tail because we have to add 1 while returning

Scanned with CamScanner

Which path should I follow?

Iterative
Recursive

we will discuss advantage & disadvantage of recursion over iteration.

Ans: every recursive program can be converted/modified into an iterative program but recursive programs are more elegant and require relatively less lines of code.

Eg: Factorial of a number.

Iterative

```
int fact(int n){
  int res=1;
  while(n!=0){
    res=res*n;
    n--;
  }
  return res; }
int main(){
  printf("%d", fact(5));
  return 0;
}
```

Recursive

```
int fact(int n){
  if(n==1)
    return 1;
  else return n*fact(n-1);
}
int main(){
  printf("%d", fact(5));
  return 0; }
```

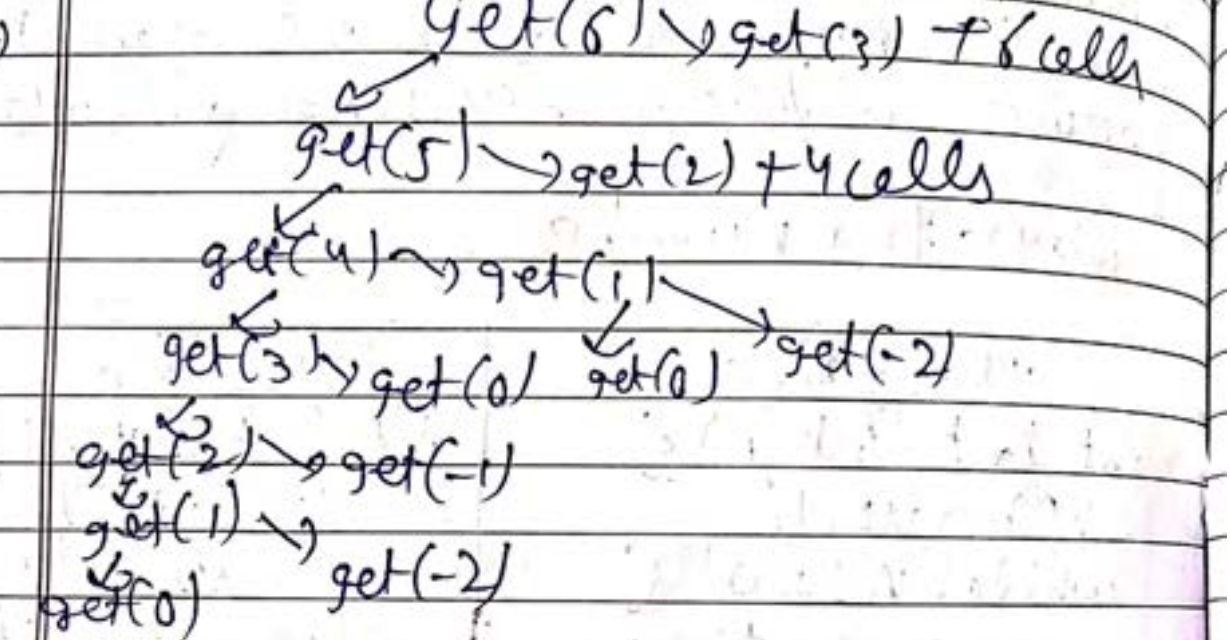
Advantage = Recursive programs require more stack space or simply space that iterative programs.

Scanned with CamScanner

(Nov 2015)

Q void get(int n){ if(n<1) return;
get(n-1); get(n-3); printf("%d", n);}
If get(6) function is being called in main() then how many time will the get() function be invoked before returning to the main()?
a) 5 b) 25 d) 35 d) 45

Ans)



Ans 25 b)

Q Determine how many # of times star will be printed on screen

```
void fun1(int n){
int i=0;
if(n>1)
    fun1(n-1);
for(i=0; i<n; i++)
    printf("*");
}
```

a) n
b) n(n+1)/2
c) n*n
d) None

Gate 2015

Q1
```
int fun(int n){
  int x=1, k;
  if(n==1) return x;
  for(k=1; k<n; ++k)
    x = x + fun(k) * fun(n-k);
  return x;
}
```

Ans for k = 1
x = 1 + fun(1) * fun(4)

fun(5) = 1 + fun(1) * fun(4) + fun(2) * fun(3)
+ fun(3) * fun(2) + fun(4) * fun(1)

= 1 + 2(fun(1) * fun(4) + fun(2) * fun(3))

fun(1) = 1
fun(2) = 1 + fun(1) * fun(1) = 2
fun(3) = 1 + fun(1) * fun(2) + fun(2) * fun(1)
= 5
fun(4) = 1 + fun(1) * fun(3) + fun(2) * fun(2)
+ fun(3) * fun(1)
= 15

fun(5) = 1 + 2(15 + 10) = 51 (C)

Scanned with CamScanner

Gate 2014

Q→ int f(int j){
static int i=50;
int k;
if(i==j){
printf("something");
k=f(i);
return 0;
}
else return 0;
}

Which is TRUE?
a) The function returns 0 for all j
b) Function prints the string "something" for all values of j
c) The function returns 0 when j=50
d) function will exhaust the runtime stack or run into an infinite loop when j=50

Gate 2016

Q→ What will be o/p of following Pseudo code when parameters are passed by reference & dynamic scoping is assumed?

```
a=3
void n(x){
x=x*a;
print(x);
}
void m(y){
a=1;
a=y-a;
n(a);
print(a);
}
void main(){
m(a);
}
```

a) 6, 2
b) 6, 6
c) 4, 2
d) 4, 4

Ans a) ∵ in m(y) a=1, a=2-1=2
in n(x) x=2×2=4 print(4)
in m(y) Print(a)
4 not 2 bcz 4 ∵ passed by reference

Scanned with CamScanner

PAGE NO. 385
DATE: / /

Q) what will be o/p of

```
void count (int n){
  static int d=1;
  printf("%d", n);
  printf("%d", d);
  d++;
  if(n>1) count(n-1);
  printf("%d", d);
}
int main(){
  count(3);
}
```

a) 3 2 2 1 3 4 4
b) 3 1 2 1 2 1 2 2
c) 3 1 2 2 1 3 4
d) 3 1 2 1 1 1 2

Q) Find true?
a) Static functions are global functions
b) Static // are related to files where they are declared
c) No concept of static functions
d) None

Q) Find true. In C, it is mandatory to declare a function before use?
a) True
b) False

In C, parameters are always
a) Passed by value b) Passed by reference c) Both d) None

Parameters are always passed by value. Can be referred is simulated in C by explicitly passing pointer value.

Scanned with CamScanner

Q. Assuming int size is 4 bytes what is going to happen when following program run?

```
#include<stdio.h>
int main(){
  printf("Hi!\n");
  main();
  return 0;
}
```

a) We can't use main inside main
b) 'Hi!' will be printed 2147483647 (i.e. $2^{31}-1$) times
c) 'Hi!' would be printed until stackoverflow
d) 'Hi!' would be printed only once

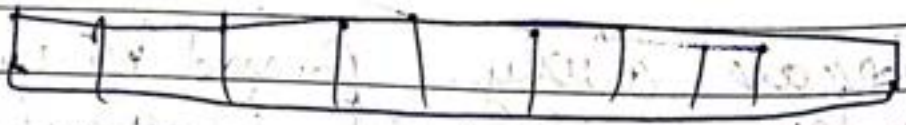
Imp-1 Break can be used in switch but looping cannot be

Basics of Arrays

Definition of Array- An array is a data structure containing a number of data values (all of which are of same type).

1) Data structure is a format for organizing & also, each data structure is designed to organize data to suite a specific purpose

Eg: Array is a data structure which you can visualize as follows:



Imagine an array as a large chunk of memory divided into smaller blocks of memory and each block is capable of holding data of same type

2. This array consists of 10 data values.

a	5	6	10	13	56	76	1	2	4	8

This array consists of 5 values.

a	5	6	10	13	56

3) Data type of array elements are same.

a	1	2	3	4	5	6

✓

b	'b'	'b'	'c'	'd'	'e'

✓

c	'a'	'b'	1	2	5.6	7.8

✗

One-d array -
The simplest form of array one can imagine is one-d array.

a	5	6	12	3	11

Declaration & def" of 1D array

Syntax: data_type name of array [no. of elements];

Ex: int arr[5]; is declaration of an

arr					

We are also defining it which means - Compiler will allocate a contiguous block of memory of size = 5 * sizeof(int)

Scanned with CamScanner

int arr[5.67]^

PAGE NO.:
DATE: / /

↳ The length of an array can be specified by any (+ve) integer constant expression

1. int arr[5];
2. int arr[5+5];
3. int arr[5*3];
4. X int arr[-5];
5. int a; int arr[a=2/3];

Best Practice - Specifying the length of an array using macros is considered to be an excellent practice

#define N 10
int arr[N];

But why?

To access an array element, write:
array name [index]

index starts from 0 and goes to n-1

int elem = arr[0]

Scanned with CamScanner

// without macro
#include<stdio.h>
int main(){
int a[10],i;
for(i=0;i<10;i++){
printf("enter the input for index %d:",i);
scanf("%d",&a[i]);
}
printf("\nArray elements are as follows:\n");
for(i=0;i<10;i++){
printf("%d",a[i]);
}
return 0;
}
suppose in future we want to ~~print~~ have 15 instead of 10, so we replace at 3 places.

// with macro
#include<stdio.h>
#define N 10
int main(){
int a[N];i;

}

future 10->15, change at 1 place only

Initializing 1D array

Method 1: `arr[5] = {1, 2, 3, 4, 5};`

Method 2: `arr[] = {1, 2, 5, 4, 3};`

Method 3:
```
int arr[5];
arr[0] = 1;
arr[1] = 2;
arr[2] = ?
arr[3] = 4;
arr[4] = 5;
```

Method 4:
```
int arr[5];
for(i=0; i<5; i++){
    scanf("%d", &arr[i]);
}
```

Method 3, 1 not preferred

Q- What if # of elements are lesser than the length specified?

eg: `int arr[10] = {45, 6, 2, 78, 5, 6};`

Ans- The remaining locations of the array are filled by value 0.

i.e `int arr[10] = {45, 6, 2, 78, 5, 6, 0, 0, 0, 0};`

Tip → Instead of writing
```
int arr[10];
for(i=0; i<10; i++)
arr[i] = 0
```
write `int arr[10]={0};`

Scanned with CamScanner

- Why not arr[10] = {};
- If it is illegal, you must have to specify at least 1 element. It can't be wordlessly empty
- It is also illegal to add more elements than length of array

int arr[5] = {1, 2, 3, 4, 5, 6}; ✗

Sometimes, we want something like this

int arr[10] = {1, 0, 0, 0, 0, 2, 3, 0, 0, 0};

How great it would be if I don't have to explicitly write these zeroes.

We want
- 1 in position 0
- 2 in " 5
- 3 " " 6

int arr[10] = {[0]=1, [5]=2, [6]=3};

This way of initialization is called designated initialization. And each number in square brackets is said to be a designator.

Scanned with CamScanner

Advantages

1. No need to bother about entries containing zeroes.
2. No need to bother about order of all
   for eg, int a[15] = {[0]=1, [5]=2};
   int a[15] = {[5]=2, [0]=1};
   Both are same

What if I don't mention length in designated array?
→ Designators could be any non-ve integer.
→ Compiler will deduce the length of the array from the largest designator in the list.

eg int a[] = {[5]=90, [20]=4, [1]=45, [49]=78};
size = 50

Also
int a[] = {1, 7, 5, [5]=90, 6, [8]=4}; is valid

=

int a[] = {1, 7, 5, 00, 90, 6, 0, 4};

But if there is a clash, then designated initializer will win

int a[] = {1, 2, 3, [2]=4, [6]=45}

=

int a[] = {1, 2, 4, 00, 0, 45}

Scanned with CamScanner

PAGE NO. 383
DATE :

Q- Write a Program to print the odd numbers in reverse order:
34 56 54 32 67 89 90 32 21
Assume that all these numbers are stored in an array.

Ans-
```
int main(){
int a[9]={34,56,54,32,67,89,90,32,21};
int i;
//original order
for(i=0;i<9;++i){
  printf("%d",a[i]);
}
printf("\n");
//reverse order
for(i=8;i>=0;i--){
  printf("%d",a[i]);
}
```

Q- Write a Program to check whether any of the digits in a number appear more than once:
Ex: 67827
Ans-
```
#include<stdio.h>
int main(){
  int seen[10]={0};
  int N;
  printf("Enter the number:");
  scanf("%d",&N);
```

Scanned with CamScanner

```
int rem;
while (N>0) {
    rem = N%10;
    if (seen[rem] == 1)
        break;
    seen[rem] = 1
    N = N/10;
}
if (N>0)
    printf("Yes");
else
    printf("No");
return 0;
}
```

→ Using size of operator

Ans-1 size of (name_of_arr) / size of (name_of_arr[0])
↓ size of whole array ↓ size of 1 array element

So number of elements = $\frac{\text{size of whole array}}{\text{size of 1 array element}}$

Multi-dimensional arrays

" " " can be defined as an array of arrays.

General form of declaring N-d array is as follows:

data type name_of_array[size1][size2]...[sizeN];

Eg: int a[3][4]; // 2-d array
int a[3][4][6]; // 3-d array

Size calculation

Size of multi-d array can be calculated by multiplying the size of all the dimensions.

for eg: size of a[10][20] = 10×20 = 200
memory = 200×4 = 800 bytes

size of a[4][10][20] = 4×10×20 = 800
mem = 800×4 = 3200 bytes

2-d array

Syntax: The basic form of declaring 2-D array is: data_type name_of_array[x][y];

~~Initialization~~

Initialization

Method 1 int a[2][3] = {1,2,3,4,5,6};

	0	1	2
0	1	2	3
1	4	5	6

Better method →

int a[2][3] = {{1,2,3}, {4,5,6}};

	0	1	2
0	1	2	3
1	4	5	6

How to access 2D array elements

Ans Eg a[0][1] = 1st row 2nd column

Scanned with CamScanner

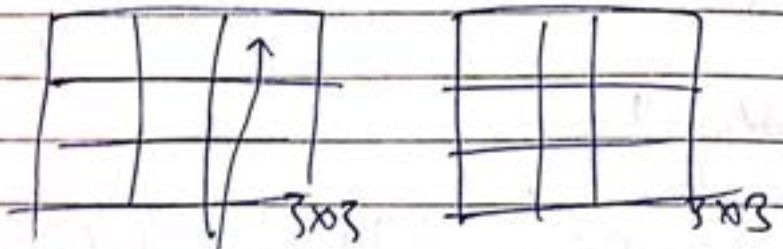
Print 2D array

```
for(int i=0; i<2; i++){
    for(j=0; j<3; j++){
        Printf("%d", a[i][j]);
    }
}
```

o/p 1 2 3 4 5 6

Visualizing 3-Darray → 2 2-D arrays of 3×3

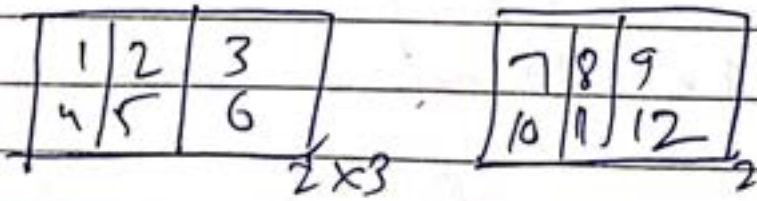
```
int arr[2][3][3];
```



arr[0][0][2]

Initialize 3D array!

Method 1 `int a[2][2][3]={1,2,3,4,5,6,7,8,9,10,11,12};`



Better Method:

```
int a[2][2][3]={{{1,2,3},{4,5,6}},
                {{7,8,9},{10,11,12}}
};
```

Scanned with CamScanner

Printing elements of 3D array

```
int a[2][2][3]={{{1,2,3},{4,5,6}},
                {{7,8,9},{10,11,12}}};
for(int i=0; i<2; ++i){
  for(j=0; j<2; j++){
    for(k=0; k<3; k++){
      printf("%d", a[i][j][k]);
}}}
```

O/P 1 2 3 4 5 6 7 8 9 10 11 12

Q. Write a program that reads a 5x5 array of integers & then prints the row sum & the column sum.

```
int main(){
int a[5][5]={
             {8, 3, 9, 0, 10},
             {3, 5, 12, 1, 1},
             {2, 8, 6, 23, 1},
             {15, 7, 3, 2, 9},
             {6, 14, 2, 6, 0},
            };
int i,j;
int sum=0;
//Row sum                    // Column sum
printf("Row sum: ");          // simply use a[j][i]
for(i=0; i<5; i++){
  for(j=0; j<5; j++){
    sum += a[i][j];
  printf("%d", sum);
  sum=0;
```

Scanned with CamScanner

Basics of matrix multiplication

Imp points

→ In order to multiply 2 matrices # of colums of 1st mtx = # of rows of 2nd mtx

→ Also size of resultant mtx = # of rows of 1st mtx X # of colums of 2nd mtx

Code →

```
#include<stdio.h>
#define MAX 50
int main(){
    int a[MAX][MAX], b[MAX][MAX]
    product[MAX][MAX];
    int arows, acolums, brows, bcolums;
    int i, j, k;
    int sum=0;
    printf("Enter the rows and colums of the
        matrix A:\n");
    scanf("%d %d", &arows, &acolums);

    printf("Enter the elements of matrix A:\n");
    for(i=0; i<arows; ++i){
        for(j=0; j<acolums; ++j)
        scanf("%d", &a[i][j]);
    }
    printf("Enter the rows & colums of matrix B:\n");
    scan("%d %d", &brows, &bcolums);
    printf("Enter the elements of matrix B:\n");
    for(i=0; i<brows; ++i){
        for(j=0; j<bcolums; ++j)
    }scanf("%d", &b[i][j]);
```

Scanned with CamScanner

```
if(brows != acolums) {
printf("Cannot Multiply \n");
}else{
printf("\n");
for(i=0; i<arows; ++i){
  for(j=0; j<bcolums; ++j){
    for(k=0; k<brows; ++k)
      sum += a[i][k] * b[k][j];
    product[i][j] = sum;
    sum=0;
  }}
printf("The resultant matrix is: \n");
for(i=0; i<arows; ++i){
  for(j=0; j<bcolums; ++j)
    printf("%6d", product[i][j]);
    printf("\n");
}}}
```

→ Constant array in C

Either one-d or multi-d arrays can be made constant by starting the declaration with the keyword const.

Eg: const int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

Can't change anywhere ahead

Advantage → It assures that the program will not modify the array which may contain some valuable information

It also helps the compiler to catch errors by informing that there is no intention to modify this array.

Scanned with CamScanner

Variable length arrays in C

vary according to user I/P, for eg

```
int n
printf("Enter the number: \n")
scanf("%d", &n);
int a[n]; ——→ Variable length array.
```

Advantages 1) At time of execution, we can decide the length of array

2) No need to choose the length while writing the code.

3) Even arbitrary expressions are possible.

eg
```
int a[3*i+5];
int a[k/7+2];
int a[i+j];
```

Points to be noted

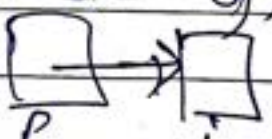
1) Variable length arrays can't have static storage duration.

2) Variable length array doesn't have the initializer.

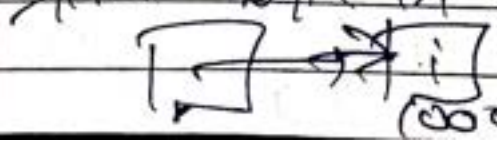
Introduction to pointers in C

Intro pointer

→ Pointer is a special variable that is capable of storing some address



→ It points to a memory location where first byte is stored



Scanned with CamScanner

Declaring & initializing

General syntax for declaring pointer variables:

data_type *pointer_name

Here data type refers to the type of the value that the pointer will point to.

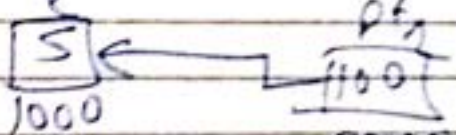
Eg: int *ptr; // Points to integer value
char *ptr; // " " Character "
float *ptr; // " " float "

Initialize

- Simply declaring a pointer is not enough
- It is important to initialize pointers before
- One way to initialize a pointer is to assign address of some variable

Eg:

int x=5
int *ptr;
ptr = &x;



=> int x=5, *ptr=&x;

Value of operator

- Value of operator/indirection operator/dereference operator is an operator that is used to access the value stored at the location pointed by the pointer.

Eg: int x=5
int *ptr;
ptr = &x;
printf("%d", *ptr);
value of operator

we can also change value of object pointed by pointer

eg *ptr = 4;

Condition - 1) Never apply indirection operator to uninitialized pointer

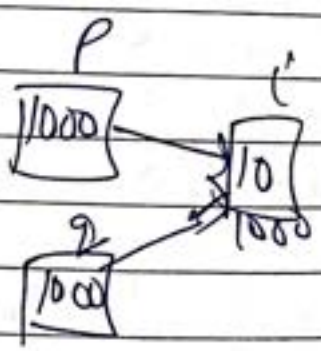
for eg: int *ptr;
printf("%d", *ptr); ✗

2) Assigning value to an uninitialized pointer is dangerous

int *ptr; ✗
*ptr = 1;
// // segmentation fault

Usually segmentation fault is caused by program trying to read or write an illegal memory location

Pointer assignment



int i = 10
int *p, *q;
p = &i;
q = p;
printf("%d %d", *p, *q); // 10 10

Note q = p is not same as *q = *p

eg int i = 0, j = 20;
int *p, *q;
p = &i; q = &j;
*q = *p

PAGE NO.: 373
DATE:

Application of pointers: Largest & smallest element in an Array

Idea

```
int a[] = {23, 45, 6, 98};
int min, max;
min = max = a[0];
for (i = 1 to 3)
    (1) If a[i] < min then
            min = a[i]
    (2) If arr[i] > max then
            max = a[i]
```

min 23 6 max 23 98

```
#include <stdio.h>
void minMax(int arr[], int len, int *min, int *max)
{
    *min = *max = arr[0];
    int i;
    for (i = 1; i < len; ++i) {
        if (arr[i] > *max)
            *max = arr[i];
        if (arr[i] < *min)
            *min = arr[i];
    }
}
int main() {
    int a[] = {23, 4, 21, 99, 987, 45, 32, 10, 123,
    986, 50, 3, 4, 5};
    int min, max;
    int len = sizeof(a) / sizeof(a[0]);
    minMax(a, len, &min, &max);
    printf("Min value is %d & max is %d", min, max);
    return 0;
}
```

Scanned with CamScanner

Returning the pointers (element at mid)

```
int main() {
  int a[] = {1, 2, 3, 4, 5};
  int n = sizeof(a)/sizeof(a[0]);
  int *mid = findMid(a, n);
  printf("%d", *mid);
  return 0;
}
int *findMid(int a[], int n){
  return &a[n/2];
}
```

or int *find

for your clarification that return type is pointer

Caution- 1) Never ever return the address of an automatic variable.

for eg:

```
int *fun() { int i=10;
return &i;   ✗
}
int main(){
  int *p = fun();
  printf("%d", *p);
}
```

Illegal : i will destroy after return &i ('i' automatic variable) and we are assigning to p address of destroyed variable

1. Consider int *p = &i;
*p = &i;

first statement is the declaration & second is simple assignment statement. why isn't in second statement, p is preceded by * symbol

In C, * symbol has different meanings depending on context in which it is used. At the time of declaration, * symbol is not acting as an indirection operator. * symbol in first statement tells compiler that p is a pointer to an integer.

But if we write *p = &i then it is wrong because here * symbol indirection operator & we can't assign the address to some integer variable.

How to print address of a variable?
→ use %p as a format specifier in printf function

```
int main() {
    int i = 10;
    int *p = &i;
    printf("The address of variable i is %p", p);
    return 0;
}
```

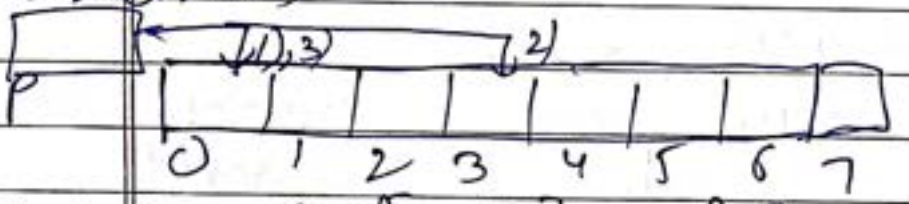
Scanned with CamScanner

Q2o/p of void fun(const int *p){*p=0;}
int main(){
const int i=10;
fun(&i);
return 0;
}
// Error.

Q4 If i is a variable & points to i. Then allowed (are)
a) *p b) **p c) &p d) *i e) *&i
Ans a), e)

Pointer Arithmetic - Addition

1) p=&a[0]



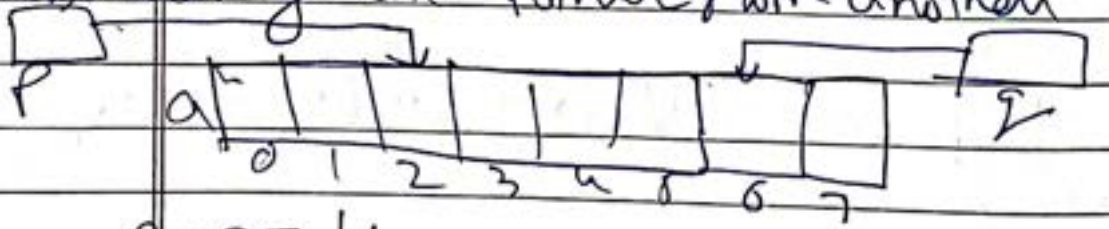
2) p=p+3 = &a[0+3] = a[3] = 3[a]

TIP-1 Don't bother about addresses until explicitly passed.

pointer Arithmetic - subtraction

3) p=p-3 = &a[3-3] = a[0]

Subtracting one pointer from another



q-p = 4
p-q = -4

Scanned with CamScanner

PAGE NO.:
DATE: / /

Undefined behaviours

```
int main() {
int i = 10;
int *p = &i;
printf("%d", *(p+3)); //Diff o/p everytime
return 0;
}
```

Performing arithmetic on pointers which are not pointing to array element leads to undefined behaviour

2) If 2 pointers are pointing to diff arrays then performing subtraction b/w them leads to undefined behaviour

```
int main(){
    int a[] = {1,2,3,4};
    int b[] = {10,20,30,40};
    int *p = &a[0];
    int *q = &b[3];
    printf("%d", q-p);
    return 0;
}
```

Pointer Arithmetic - Increment & decrement

Post increment

```
int main() {
int a[] = {5, 16, 7, 89, 45, 32, 23, 70};
int *p = &a[0];
printf("%d", *(p++)); // 5
printf("%d", *p); // 16
return 0;
}
```

Scanned with CamScanner

2) Pre increment

```
int main() {
  int a[] = { 5, 16, 7, 89, 45, 32, 23, 10 };
  int *p = &a[0];
  printf("%d", *(++p));   //16
  return 0;
}
```

3) Pre & Post Decrement

```
int main() {
  int a[] = {5, 16, 7, 8, 10};
  int *p = &a[2];
  printf("%d", *(--p));   //16
  printf("%d", *(p--));   //16
  return 0;
}
```

Comparing pointers = Use relational operators (<, >, <=, >=) & equality operators (==, !=) to compare pointers.

2) Only possible when both pointers point to same array

3) o/p depends upon the relative positions of both the pointers.

4)

```
int main() {
  int a[] = {1, 2, 3, 4, 5, 6};
  int *p = &a[3];
  int *q = &a[5];
  printf("%d\n", p<=q);   //1
  printf("%d\n", p>=q);   //0
  q = &a[3];
  printf("%d", p==q);     //1
  return 0;
}
```

Scanned with CamScanner

PAGE NO.: 59
DATE:

Sum of elements of array using pointer

```
int main(){
    int a[] = {11, 22, 36, 5, 2};
    int sum=0, *p;
    for (p=&a[0]; p<=&a[4]; p++)
        sum += *p;
    printf("Sum is %d", sum);
}
```

Using array name as a pointer

Ans: Name of an array can be used as a pointer to the first element of an array

Eg →

```
int main(){
    int a[5];
    *a = 10;
    printf("%d", a[0]) // 10
}
```

*(a+i) = a[i]

Now let's rewrite sum of elements of array

```
int main(){
    int a[] = {11, 22, 36, 5, 2};
    int sum=0, *p;
    for (p=a; p<=a+4; p++)
        sum += *p;
    Printf("Sum is %d", sum);
}
```

O/P Sum is 76

Scanned with CamScanner

Be Careful

1) It's true that we can use array names as pointers, but assigning a new address to them is not possible

for eg

```
int main() {
  int a[] = {11, 22, 36, 5, 24};
  printf("%p", a++); //error
  return 0;
}
```

∴ a = a+1, we are increasing (not possible) (we are changing base address)

Reverse a series of numbers using pointers

```
#include<stdio.h>
#define N 5
int main(){
  int a[N], *p;
  printf("Enter %d elements:", N);
  for(p=a; p<=a+N-1; p++)
    scanf("%d", p);
  printf("Elements in reverse order:\n");
  for(p = a+N-1; p>=a; p--)
    printf("%d", *p);
  return 0;
}
```

Scanned with CamScanner

Passing array name as argument to function.

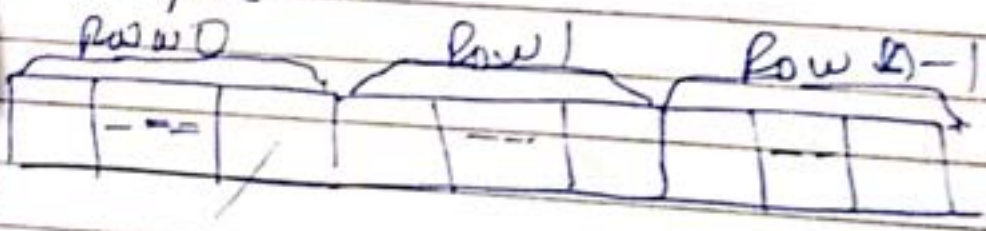
```
int add(int b[], int len){
    int sum=0,i;
    for(i=0; i<len; i++)
      sum += b[i];
    return sum;
}
int main(){
    int a[]={1, 2, 3, 4};
    int len = sizeof(a)/sizeof(a[0]);
    Printf("%d", add(a,len));
    return 0;
}
```

↓ Imp

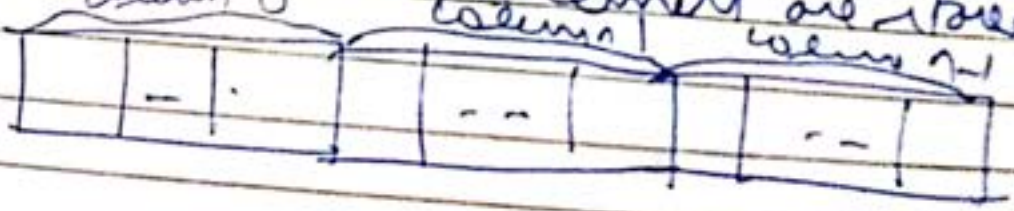
Using pointers to print 2D arrays:

1) Diff b/w Row Major & Column Major order

Row major order: Elements are stored row by row



Column major order - Elements are stored column by column.



Imp - C stores multi-d arrays in row major order

Scanned with CamScanner

Traditionally,

```
for (i=0; i<row; i++){
    for (j=0; j<col; j++){
        printf ("%d", a[i][j]);
    }}
```

Using Pointers

```
for (p=&a[0][0]; p<=&a[row-1][col-1]; p++)
    printf ("%d", *p);
```

Gate 2002 Diag

Q- Consider a[100][100]. Assuming that main memory is byte addressable and that the array is stored starting from the memory address 0, the address of a[40][50] is

a) 4050
b) 4050
c) 5040
d) 5050

Ans, Row Major

Formula - &a[i][j] = BA + [(i-lb1) × NC + (j-lb2)] × C

BA = Base address of whole 2D array
NC = # of colums
C = size of data type of elements stored in array (in bytes)
a[lb1 -- ub1][lb2 -- ub2]

Scanned with CamScanner

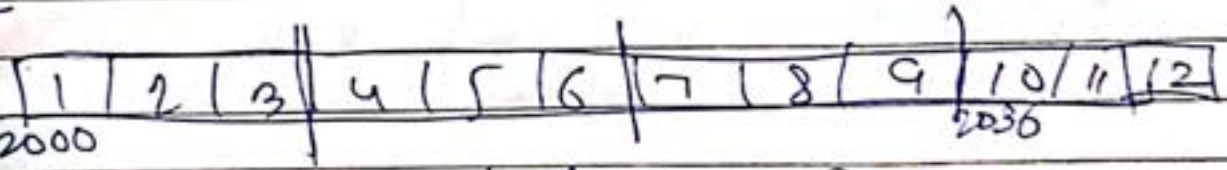
Date 205

1) What is o/p of following c code? Assume that address of x is 2000 (in decimal) and an integer requires 4 bytes of memory

```
#include<stdio.h>
int main(){
  unsigned int x[4][3]={{1,2,3},{4,5,6},
                        {7,8,9},{10,11,12}};
printf("%u, %u, %u", x+3, *(x+3), *(x+2)+3);
```

a) 2036, 2036, 2036
b) 2012, 4, 2204
c) 2036, 10, 10
d) 2012, 4, 6

Ans



x = Pointer to first 1D array
x+3 = pointer to 4th " " (2036)
*(x+3) = " " 1st element of 4th 1D (2036)
*(x+2)+3 = " " " " 3rd + 3 = 2036

Scanned with CamScanner

```
#include<stdio.h>
int main(){
 int a[5]={1,2,3,4,5};
 int *p=a;          // 1
 printf("%d", *p);
 return 0;
}
```

(*p) is a pointer to 5 integer elements

But if int (*p)[5]=&a; — (1)

Pointer to whole array

Eg: 2D array b

1	2	3	4
1000	1004	1008	1012

1000 1008

*b = pointer to 1st element of 1st 1D array

&*b = b = " " 1st 1D array

in (1) we have to put &a not a so that we get address to whole array a and not the first element

Soln: printf("%d", p); // address to whole array a
" (" , *p); // address to 1st element a
" (" *p) // value of " "

Q-1 o/p of — Illegal and mean 2 round

```
#include<stdio.h>
int main(){
 int a[][3]={1,2,3,4,5,6};
 int (*ptr)[3]=a;
 printf("%d %d", (*ptr)[0], (*ptr)[2]);
 ++ptr;
 printf("%d %d", (*ptr)[1], (*ptr)[2]);
}
```

a) 2358
b) 2346
c) 1500
d) Error

Scanned with CamScanner

*ptr)[3] - a // base address of 1st 1-D array
*ptr[1] = base address of 1st element of 1-D array
= 2
*ptr[2] = 3

*ptr = 2nd 1-D array
*ptr[1] - 2 " " 1st element = 5
*ptr[2] - " " 2nd " 6

Q O/P of

```
void f(int *p, int *q){
    p = q;
    *p = 2;
}
int i=0, j=1;
int main(){
    f(&i, &j);
    printf("%d %d \n", i, j);
}
```

a) 2 2
b) 2 1
c) 0 1
d) 0 2

date 2010
Q) What is value printed by

```
#include<stdio.h>
int f(int *a, int n){
if(n<=0) return 0;
else if(*a%2==0) return *a + f(a+1, n-1);
else return *a - f(a+1, n-1);
}
int main(){
int a[] = {12, 7, 13, 4, 11, 6};
printf("%d", f(a, 6));
getchar();
return 0;
}
```

a) -9
b) 5
c) 15
d) 19

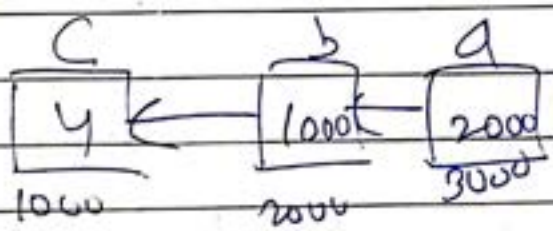
Scanned with CamScanner

Code snippet

```
Q- int f(int x, int *py, int **ppz){
   int y, z;
   **ppz += 1;
   z = **ppz;
   *py += 2;
   y = *py;
   x += 3;
   return x+y+z; }
void main(){
   int c, *b, **a;
   c = 4, b = &c, a = &b;
   printf("%d", f(c, b, a));
}
```

a) 18
b) 19
c) 21
d) 22

Ans



**a means it is a pointer to the pointer. It consist contain address of a variable but pointer (always).

*a means go to b

*b " " " c

So **ppz += 1; // 5

z = 5

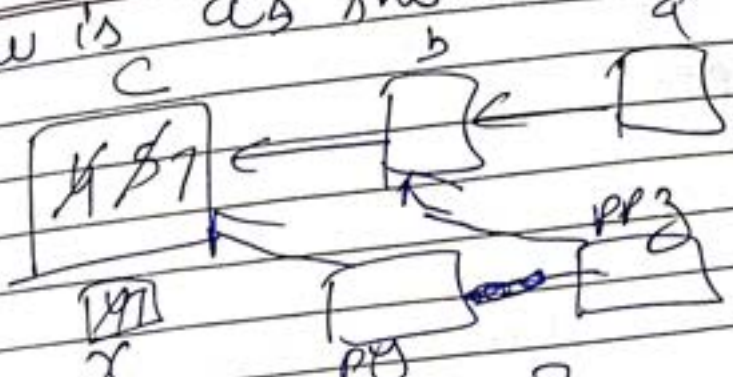
y = 7

x = 7

Ans = 19

Scanned with CamScanner

Flow is as shown



x=7, y=5, y=7

Q-1 which one of the choices given below would be printed

```
void swap(int *x, int *y){
static int *temp;
temp = x;
x = y;
y = temp;
}
void printab(){
static int i, a=3, b=6;
i=0;
while(i<=4){
if((i++)%2==1) continue;
a = a+i;
b = b+i;
}
swap(&a, &b);
printf("a=%d, b=%d\n", a, b);
}
main(){
printab();
printab();
}
```

a) a=0, b=3
a=0, b=3
b) a=3, b=0
a=12, b=9
c) a=3, b=6
a=3, b=6
d) a=6, b=3
a=15, b=12

Scanned with CamScanner

i	a	b
~~0~~	~~3~~ 6	~~6~~ 3

i=0

if((i++)%2==0) continue;

So a+=i, a=-2

b=-5

Ily next time a+=i, then i=3

" " " " " i=5

Now swap (no use) we are swaping the addresses not values

So O/P a=6, b=3

2nd time also static variable won't be initialized again.

So static int i, a=-3, b=-6; //House

i=0

i	a	b
0	~~6~~ 15	~~3~~ 12

O/P a=15, b=12

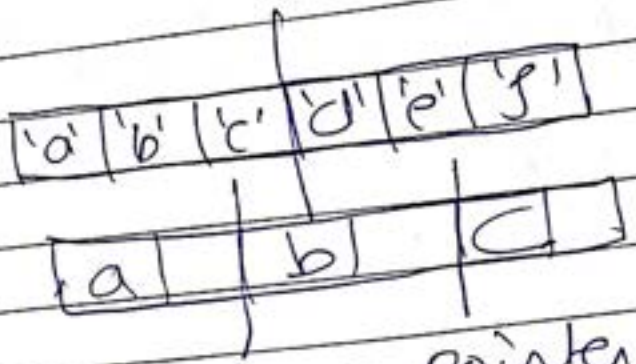
Q) Contents of b at end

```
#include<stdio.h>
int main(){
int i,j;
char a[2][3]={{'a','b','c'},
{'d','e','f'}};
char b[3][2];
char *p=*b;
```

Scanned with CamScanner

```
for(i=0; i<2;++i){
  for(j=0;j<3;++j){
    *(p+2*j+i)=a[i][j]
  }
}
```

a b b) a d c) a c d) a e
c d b e e b d c
e f c f d f b f



char *p = b = pointer to start to b
char *p = *b = " " 1st element of b

i=0 j=0 'a' at *p
i=0 j=1 'b' at *(p+2)
i=0 j=1 'c' at *(p+4) (b)
// y next

ing literals: String literal (or String constant) is a sequence of characters enclosed within double quotes.

eg "Hello everyone" "Hola a todos"
↑ n spaces

% is a placeholder.
always double quotes

What if we want to print in next line

a) printf("%s", "You have to dream before it comes true.
—A.P.J. Abdul Kalam"); // Error

b) printf("%s", "You have to dream before it comes true.
—A.P.J Abdul Kalam"); // No Error

In C, this is called "Splicing"

c) printf("%s", "You have to dream before it comes true."
"A.P.J Abdul Kalam");

a) Error
b) Considered 4 spaces before --

You have to dream before it comes true. --A.P.J Ab--

c) You have to .. .. --A.P.J Ab--

Disadv :- this space is also printed (4 space here)

To remove this disadv just give no space

Here indentation is considered

printf("Earth");

"Earth" is a string literal.

Argument to printf & scanf function is always a string literal

But, what we are actually passing to the printf / scanf?

Let's understand how string literals are stored?

String literals are stored as an array of characters.



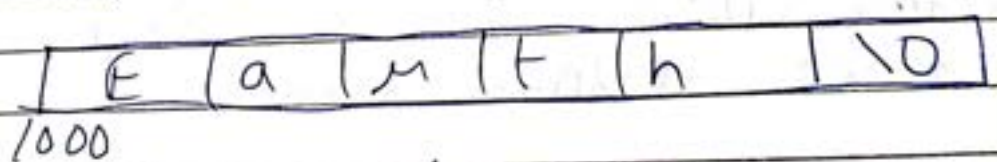
Total 6 bytes of read-only memory is allocated to the above string literal.

'\0' character must not be confused with '0' character.

Both have different ASCII codes. '\0' has the code 0 while '0' has code 48.

But what we are actually passing to printf/scanf?

→ In C, compiler treats a string literal as a pointer to the first character.



So to the printf or scanf, we are passing a pointer to the first character of a string literal.

Both printf & scanf functions expecting character pointer (char *) as an argument.

Scanned with CamScanner

passing "Earth" is equivalent to passing the pointer to letter 'E'

Performing operations on string literals

1) Assigning string literal to a pointer
char *ptr = "Hello World!!";

ptr contains the address of the first character of the string literal.

∴ we can subscript it to get some character of the string literal.

"Hello"[0] ⇒ 'H'
" [1] ⇒ 'e'
" [2] ⇒ 'l'
" [3] ⇒ 'l'
" [4] ⇒ 'o'

Point to be noted :
⇒ String literal can't be modified. It causes undefined behaviour.

e.g. char *ptr = "Hello";
*ptr = 'M'; // Not allowed

String literals are also known as string constants. They have been allocated a read only memory. So we cannot alter them but character pointer itself has been allocated read write memory. So the same pointer can point to some other string literal.

Scanned with CamScanner

String literal Vs character constant

String literal & character constant are not same, for eg "A" ≠ 'A'

↑ represented by a pte to char 'A' — represented by an integer (ASCII: 72)

So printf("\n") ~ puts("\n"); x error

Declaring & Initializing a String Variable

Declaring => A string variable is a one-dimensional array of characters that is capable of holding a string at a time.

for eg. char s[6];

Note: Always make array one character longer than the string. If length of the string is 5 characters long then don't forget to make extra room for Null character.

Failing to do the same may cause unpredictable results when program is executed as some C libraries assume the strings are null terminated

constants only often allocated RO

Initializing
Eg char s[6] = "Hello";

Although, it seems like "Hello" in above eg is a string literal but it is not.
When a string is assigned to a character array, then this character array is treated like other types of array. We can modify it's characters.

char s[6] = "Hello";

char s[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

Recall: We can't modify a string literal

```
char *ptr = "Hello";
*ptr = 'M';
```

But we can modify a char array

```
char s[6] = "Hello";
s[0] = 'M';
// No problem.
```

Short length Initializer
char s[7] = "Hello";

| H | e | l | l | o | \0 | \0 |

Long length initializer
char s[4] = "Hello" // Not allowed / Error
→ Hele → Rest part truncated
O/P Hell ? Warning: Initializer string for array of char is too

Scanned with CamScanner

Equal length in both lines

char s[5] = "Hello"; // Not allowed/Bad

s | H | e | l | l | o | ↑ No '\0' at end

Omitting the length

char s[] = "Hello";

Automatically compiler sets aside 6 characters for s which is enough to store the string "Hello" with null character.

Writing string using printf

char *ptr = "Hello World";
printf("%s", ptr);

"%.ns" is used to print just a part of string where n is number of characters to be displayed on the screen.

"%m.ns" is used to print just a part of string where n is number of characters to be displayed & m denotes the size of the field within which the string will be displayed

char *ptr = "Hello World!";
printf("%.5s", ptr); // Hello
printf("%6.5s", ptr); // Hello
(space)

	H	e	l	l	o
m=6					

Write string using puts

puts() function is declared in <stdio.h> library & is used to write strings to the o/p screen.

Also, puts() function automatically writes a newline character after writing the string to the o/p screen.

```
char *s = "Hello";
puts(s);
puts(s);
```

o/p
Hello, newline
Hello, newline
newline

Reading strings using scanf & gets

Reading strings using scanf - Using scanf(), we can read a string into a string variable (character array)

```
char a[10];
printf("Enter the string: \n");
scanf("%s", a); ← Like any array name, a is treated as a pointer to the first element of the array, hence there is no need to use &
printf("%s", a);
```

I/P: You are most welcome
O/P: You

Why only You?
→ scanf() doesn't store the white space characters in the string variable & only reads characters other than white spaces & store them in the specified character array until it encounters a space, tab, ↵ (enter)

Scanned with CamScanner

Reading String using gets

In order to read an entire line of input, gets() function can be used.

#include

```
char a[10];
printf("Enter the string:\n");
gets(a);
printf("%s", a);
```

I/P: You are most welcome
O/P: Only 10 character

Both, gets() & scanf() functions have no way to detect when the character array is full.
Both of them never checks the maximum limit of I/P characters. Hence, they may cause undefined behaviour & basically lead to buffer overflow error which eventually causes the program to crash.

Though scanf() has the way to set the limit for number of characters to be stored in the character array.
By using %ns where n indicates number of characters allowed to store in the character array.

```
char a[10];
printf("Enter the string:\n");
scanf("%9s", a);
printf("%s", a);
```

I/P: You are most welcome
O/P: You are mos

But unfortunately, gets() is called UNSAFE. It will try to write the characters beyond the memory allocated to the character array which is unsafe because it will simply overwrite the memory beyond the memory allocated to the character array.

Hence, it is advisable to not use the gets() function.

Scanned with CamScanner